



WYDZIAŁ ELEKTRONIKI,
TELEKOMUNIKACJI
I INFORMATYKI

Imię i nazwisko studenta: Wiktor Czetyrbok
Nr albumu: 188695
Poziom kształcenia: studia drugiego stopnia
Forma studiów: stacjonarne
Kierunek studiów: Informatyka
Specjalność: Inżynieria systemów informacyjnych

PRACA DYPLOMOWA MAGISTERSKA

Tytuł pracy w języku polskim: Analiza porównawcza wydajności, skalowalności i kosztów utrzymania architektury monolitycznej z architekturą opartą na mikroserwisach w środowiskach chmurowych

Tytuł pracy w języku angielskim: A comparative analysis of the performance, scalability, and maintenance costs of monolithic and microservice architectures in cloud environments

Opiekun pracy: dr Adam Przybyłek

Streszczenie

Celem pracy była analiza porównawcza architektury monolitycznej oraz architektury mikroserwisowej pod kątem ich wydajności, skalowalności oraz kosztów utrzymania w środowiskach chmurowych. W dobie rosnącej złożoności systemów informatycznych oraz dynamicznego przyrostu liczby użytkowników coraz większe znaczenie zyskują modele architektoniczne umożliwiające efektywne zarządzanie zasobami, elastyczność oraz niezawodność. W pracy zbadano, w jaki sposób wybór architektury wpływa na realne parametry działania aplikacji, a także jakie konsekwencje niesie dla organizacji pod względem operacyjnym i finansowym.

Przeprowadzony przegląd literatury pozwolił zidentyfikować aktualne podejścia do pomiaru wydajności i skalowalności systemów rozproszonych oraz istniejące narzędzia benchmarkingowe. Jednak większość dostępnych badań analizuje architektury w oderwaniu od praktycznego kontekstu wdrożenia w chmurze, co wskazało na istnienie luki badawczej — brak kompleksowego porównania monolitu i mikroserwisów uruchomionych w porównywalnym środowisku, z zastosowaniem jednolitych metod obciążenia i identycznych scenariuszy testowych.

W ramach pracy zaimplementowano aplikację benchmarkingową w dwóch wariantach architektonicznych — monolitycznym oraz mikroserwisowym — oraz wdrożono oba modele na platformie chmurowej. Zastosowano testy obciążeniowe generowane za pomocą dedykowanego narzędzia oraz przeprowadzono analizę skalowania pionowego i poziomego. Dodatkowo zbadano wpływ konfiguracji infrastruktury, mechanizmów komunikacji, wielowątkowości oraz sposobu zarządzania zasobami na ogólną efektywność aplikacji.

Wyniki badań wskazują, że architektura monolityczna charakteryzuje się wyższą wydajnością przy niewielkim obciążeniu oraz niższymi kosztami utrzymania w prostych instalacjach, natomiast architektura mikroserwisowa oferuje znacząco lepszą skalowalność poziomą, niezależność komponentów oraz większą elastyczność w zakresie optymalizacji wybranych obszarów systemu. Jednocześnie mikroserwisy wprowadzają koszty związane z komunikacją między usługami, złożonością infrastruktury oraz większym nakładem na utrzymanie.

Praca kończy się omówieniem ograniczeń badania, wkładu pracy w obszar inżynierii oprogramowania oraz propozycją kierunków dalszych badań, obejmujących m.in. analizę architektur hybrydowych, automatyczne skalowanie zasobów oraz wpływ technologii serverless na koszty operacyjne aplikacji.

Słowa kluczowe: Informatyka, Mikroserwisy, Monolit, Analiza kosztów, GCP, gRPC, Aplikacja webowa

Dziedzina nauki i techniki, zgodnie z wymogami OECD: Nauki o komputerach i informatyka

Abstract

The aim of this thesis was to conduct a comparative analysis of monolithic and microservice architectures in terms of performance, scalability, and maintenance costs in cloud environments. As software systems become increasingly complex and the number of users continues to grow, architectural models that provide effective resource management, flexibility, and reliability gain significant importance. This work investigates how the choice of software architecture influences real application performance parameters and what operational and financial consequences it entails for an organization.

A systematic literature review was carried out to identify current approaches to measuring the performance and scalability of distributed systems, as well as existing benchmarking tools. However, most available studies examine these architectures without a practical cloud deployment context, exposing a research gap — the lack of a comprehensive comparison of monolithic and microservice applications deployed in equivalent environments, using unified load-testing methods and identical test scenarios.

As part of the thesis, a benchmarking application was implemented in two architectural variants — monolithic and microservice — and both versions were deployed on a cloud platform. Load tests were conducted using a dedicated tool, and vertical and horizontal scaling were analyzed. Additionally, the impact of infrastructure configuration, communication mechanisms, multithreading, and resource management strategies on overall application efficiency was examined.

The study results indicate that monolithic architecture provides higher performance under low load and lower maintenance costs in simpler deployments. In contrast, microservice architecture offers significantly better horizontal scalability, component independence, and greater flexibility in optimizing specific parts of the system. At the same time, microservices introduce additional costs related to inter-service communication, infrastructure complexity, and operational overhead.

The thesis concludes with a discussion of study limitations, the scientific contribution of the work, and directions for further research, including hybrid architectural models, automatic resource scaling, and the impact of serverless technologies on the operational costs of cloud applications.

Keywords: Computer Science, Microservices, Monolith, Cost Analysis, GCP, gRPC, Web Application

OECD Field of Science and Technology Classification: Computer and Information Sciences

SPIS TREŚCI

WYKAZ WAŻNIEJSZYCH OZNACZEŃ I SKRÓTÓW	6
1. WSTĘP	7
1.1. Motywacja, cel pracy oraz pytania badawcze	7
1.2. Struktura pracy	8
2. PODSTAWY TEORETYCZNE ARCHITEKTUR SYSTEMÓW	10
2.1. Architektura monolityczna	10
2.1.1. Cechy charakterystyczne architektury monolitycznej	10
2.1.2. Zalety i wady architektury monolitycznej	10
2.2. Architektura mikroservisowa	11
2.2.1. Cechy charakterystyczne architektury mikroservisowej	11
2.2.2. Zalety i wady architektury mikroservisowej	12
2.2.3. Metody komunikacji między mikroservisami	12
2.3. Skalowalność i wydajność systemów rozproszonych	13
2.4. Koszty utrzymania aplikacji w środowiskach chmurowych	13
2.5. Podsumowanie	14
3. SYSTEMATYCZNY PRZEGLĄD LITERATURY	15
3.1. Cel i zakres przeglądu	15
3.2. Pytania badawcze	15
3.3. Metodyka przeglądu	16
3.4. Strategia wyszukiwania	16
3.5. Źródła danych i zakres przeszukiwania	17
3.6. Kryteria włączenia i wyłączenia	18
3.6.1. Kryteria włączenia	18
3.6.2. Kryteria wyłączenia	18
3.7. Proces selekcji publikacji	18
3.8. Ekstrakcja i synteza danych	19
3.9. Analiza badań dotyczących porównania architektury monolitycznej i mikroservisowej	20
3.9.1. Porównania pod względem wydajności	20
3.9.2. Porównania pod względem skalowalności	21
3.9.3. Porównania pod względem kosztów utrzymania	21
3.10. Luki badawcze i uzasadnienie własnego podejścia	21
3.11. Podsumowanie	22
4. IMPLEMENTACJA APLIKACJI BENCHMARKOWEJ	23
4.1. Cel rozdziału	23
4.2. Charakterystyka aplikacji benchmarkowej	23
4.3. Model wejścia i wyjścia systemu	24
4.4. Logika przetwarzania obrazu	25
4.5. Implementacja wariantu monolitycznego	25
4.6. Implementacja wariantu mikroservisowego	26
4.7. Stosy technologiczne	29
4.7.1. Java i Spring	29
4.7.2. JavaScript i Node.js	29
4.7.3. Python i Flask	30
4.8. Dane testowe i przygotowanie środowiska	30
4.9. Znaczenie przyjętej implementacji dla badań eksperymentalnych	30

4.10 Podsumowanie	31
5. METODYKA BADAŃ EKSPERYMENTALNYCH	32
5.1. Cele i scenariusze testowe	32
5.2. Narzędzia wykorzystane do pomiarów	32
5.3. Metryki oceny	33
5.3.1. Przepustowość	33
5.3.2. Latencja	33
5.3.3. Zużycie zasobów obliczeniowych	33
5.3.4. Koszty utrzymania	34
5.4. Scenariusz testów bez skalowania	34
5.5. Scenariusz testów skalowania wertykalnego	34
5.6. Scenariusz testów skalowania horyzontalnego	35
5.7. Sposób gromadzenia i przetwarzania wyników	35
5.8. Ograniczenia metodyki badawczej	36
6. WYNIKI BADAŃ I ICH ANALIZA	37
6.1. Wyniki skalowania wertykalnego	37
6.2. Wyniki skalowania horyzontalnego	39
6.3. Porównanie skalowania wertykalnego i horyzontalnego	40
6.4. Podsumowanie wyników	42
7. ZAGROŻENIA DLA RZETELNOŚCI WYNIKÓW	43
7.1. Zagrożenia wewnętrzne	43
7.2. Zagrożenia zewnętrzne	43
7.3. Zagrożenia konstrukcyjne	44
7.4. Zagrożenia związane z trafnością wnioskowania	45
8. PODSUMOWANIE I WNIOSKI	46
8.1. Odpowiedzi na pytania badawcze	46
8.2. Najważniejsze wnioski dotyczące wydajności	46
8.3. Najważniejsze wnioski dotyczące skalowalności	47
8.4. Najważniejsze wnioski dotyczące kosztów utrzymania	47
8.5. Ograniczenia pracy	48
8.6. Możliwe kierunki dalszych badań	48
WYKAZ LITERATURY	50
WYKAZ RYSUNKÓW	51
WYKAZ TABEL	52
WYKAZ LISTINGÓW	53

WYKAZ WAŻNIEJSZYCH OZNACZEŃ I SKRÓTÓW

- VM - Virtual Machine
- IT - Information Technology
- GCP - Google Cloud Platform
- API - Application Programming Interface
- ID - Identification
- CI/CD - Continuous Integration / Continuous Deployment
- REST - Representational State Transfer
- CSV - Comma-separated Values

1. WSTĘP

1.1. *Motywacja, cel pracy oraz pytania badawcze*

W ostatnich latach systemy informatyczne są coraz częściej projektowane i wdrażane w środowiskach chmurowych, w których kluczowe znaczenie mają wydajność, skalowalność oraz koszty utrzymania aplikacji. Wśród najczęściej rozważanych podejść architektonicznych architektura monolityczna oraz architektura oparta na mikroserwisach stanowią dwa zasadniczo odmienne sposoby budowy i eksploatacji systemów informatycznych. Architektura monolityczna jest często postrzegana jako prostsza w implementacji, wdrażaniu i utrzymaniu w przypadku mniejszych systemów, natomiast podejście mikroserwisowe jest powszechnie kojarzone z większą modułowością, elastycznością i możliwością skalowania. Korzyści te nie są jednak osiągalne bez pewnych kompromisów.

Rosnąca popularność mikroserwisów sprawiła, że wiele organizacji decyduje się na stosowanie tego stylu architektonicznego, często przy założeniu, że jest on z natury lepszy od architektury monolitycznej, zwłaszcza w środowiskach chmurowych. W praktyce rzeczywiste korzyści wynikające z zastosowania mikroserwisów zależą jednak w dużym stopniu od charakteru aplikacji, rodzaju obciążenia, modelu wdrożenia oraz narzutu operacyjnego. Zwiększona złożoność komunikacji między usługami, orkiestracji, monitorowania oraz zarządzania infrastrukturą może w istotny sposób wpływać zarówno na wydajność systemu, jak i na całkowity koszt jego utrzymania.

Jednocześnie platformy chmurowe oferują mechanizmy takie jak skalowanie wertykalne i horyzontalne, konteneryzacja oraz automatyzacja infrastruktury, które mogą wpływać na efektywność obu stylów architektonicznych. W rezultacie na pytanie, czy architektura monolityczna czy mikroserwisowa sprawdza się lepiej w środowisku chmurowym, nie można udzielić jednoznacznej, uniwersalnej odpowiedzi. Wymaga to przeprowadzenia systematycznego porównania w kontrolowanych warunkach eksperymentalnych, z uwzględnieniem nie tylko podstawowych metryk wydajnościowych, ale również cech skalowalności oraz kosztów utrzymania.

Mimo że w literaturze przedmiotu można znaleźć liczne opracowania dotyczące wzorców architektonicznych oraz systemów chmurowych, nadal brak jednoznacznego konsensusu co do rzeczywistego wpływu wybranego stylu architektonicznego na wydajność, skalowalność i efektywność kosztową aplikacji wdrażanych w chmurze. Wiele prac koncentruje się wyłącznie na wybranych atrybutach jakościowych, natomiast stosunkowo niewiele przedstawia kompleksowe porównanie implementacji monolitycznej i mikroserwisowej testowanych w porównywalnych warunkach.

Z tego względu główną motywacją niniejszej pracy jest zbadanie, w jaki sposób wybór pomiędzy architekturą monolityczną a architekturą opartą na mikroserwisach wpływa na działanie aplikacji wdrożonej w środowisku chmurowym. Analiza została ukierunkowana na trzy kluczowe

aspekty: wydajność, skalowalność oraz koszty utrzymania. W celu rozwiązania tego problemu porównane zostaną dwie funkcjonalnie równoważne implementacje tej samej aplikacji: jedna oparta na architekturze monolitycznej, a druga na architekturze mikroserwisowej. Oba warianty zostaną wdrożone w środowisku chmurowym i poddane ocenie z wykorzystaniem zdefiniowanej metodyki benchmarkowej.

Głównym celem pracy jest przeprowadzenie analizy porównawczej wydajności, skalowalności oraz kosztów utrzymania architektury monolitycznej i architektury opartej na mikroserwisach w środowiskach chmurowych. Aby osiągnąć ten cel, przygotowane zostanie środowisko eksperymentalne, zdefiniowane zostaną scenariusze pomiarowe, a uzyskane wyniki zostaną przeanalizowane w celu wskazania mocnych i słabych stron obu podejść.

W pracy sformułowano następujące główne pytanie badawcze:

W jaki sposób wybór pomiędzy architekturą monolityczną a architekturą opartą na mikroserwisach wpływa na wydajność, skalowalność oraz koszty utrzymania aplikacji wdrożonej w środowisku chmurowym?

W celu bardziej szczegółowej oceny rozważane są również następujące pytania pomocnicze:

- Jak oba style architektoniczne różnią się pod względem przepustowości i czasu odpowiedzi wraz ze wzrostem obciążenia?
- Jak skutecznie aplikacje monolityczne i mikroserwisowe wykorzystują skalowanie wertykalne i horyzontalne w środowisku chmurowym?
- Jaki wpływ ma wybrany styl architektoniczny na wykorzystanie zasobów infrastrukturalnych oraz koszty utrzymania?
- W jakich warunkach jeden styl architektoniczny staje się bardziej korzystny od drugiego?

Wyniki przeprowadzonych badań mogą mieć znaczenie zarówno poznawcze, jak i praktyczne. Z perspektywy naukowej praca wnosi wkład do dyskusji dotyczącej rzeczywistych zalet i ograniczeń współczesnych wzorców architektonicznych. Z perspektywy praktycznej może stanowić wsparcie dla architektów oprogramowania oraz inżynierów przy podejmowaniu bardziej świadomych decyzji związanych z wyborem stylu architektonicznego dla systemów wdrażanych w środowiskach chmurowych.

1.2. Struktura pracy

Praca została podzielona na kilka głównych etapów obejmujących analizę literatury, projekt i implementację środowiska badawczego, przygotowanie metodyki eksperymentalnej, przeprowadzenie testów wydajnościowych oraz interpretację uzyskanych wyników.

Pierwszy etap obejmował zdefiniowanie problemu badawczego, określenie motywacji do podjęcia tematu oraz przegląd literatury dotyczącej architektury monolitycznej, architektury mikroserwisowej, środowisk chmurowych, mechanizmów skalowania oraz metod oceny wydajności systemów. Etap ten pozwolił na wskazanie luki badawczej oraz sformułowanie głównego pytania badawczego i pytań pomocniczych.

Drugi etap koncentrował się na zaprojektowaniu i implementacji środowiska eksperymentalnego. Przygotowano dwa funkcjonalnie równoważne warianty analizowanej aplikacji: wariant monolityczny oraz wariant mikroserwisowy. Dodatkowo skonfigurowano infrastrukturę chmurową, proces wdrażania oraz mechanizmy automatyzacji niezbędne do przeprowadzania powtarzalnych eksperymentów.

Trzeci etap obejmował opracowanie metodyki badawczej. Na tym etapie określono metryki oceny, przygotowano scenariusze testowe, wybrano narzędzia do generowania obciążenia i monitorowania systemu oraz zdefiniowano strategie skalowania. Szczególną uwagę poświęcono zapewnieniu porównywalnych warunków testowych dla obu wariantów architektonicznych.

Kolejny etap polegał na przeprowadzeniu eksperymentów benchmarkowych w środowisku chmurowym. Testy wykonano dla różnych poziomów obciążenia oraz różnych konfiguracji skalowania w celu oceny zachowania obu implementacji pod względem przepustowości, latencji, skalowalności oraz wykorzystania zasobów. Uwzględniono również aspekty związane z kosztami utrzymania obu rozwiązań.

Ostatni etap obejmował analizę i interpretację zebranych wyników, wskazanie najważniejszych różnic pomiędzy badanymi architekturami, omówienie ograniczeń przeprowadzonego badania oraz sformułowanie wniosków końcowych. Wnioski te stanowią podstawę do udzielenia odpowiedzi na postawione pytania badawcze oraz wskazania możliwych kierunków dalszych badań.

2. PODSTAWY TEORETYCZNE ARCHITEKTUR SYSTEMÓW

2.1. *Architektura monolityczna*

Architektura monolityczna jest jednym z najstarszych i najdłużej stosowanych sposobów projektowania systemów informatycznych. W tym podejściu cała aplikacja tworzona jest jako jeden spójny system, który obejmuje interfejs użytkownika, logikę biznesową, warstwę dostępu do danych oraz mechanizmy integracyjne. Poszczególne elementy systemu mogą być wewnętrznie rozdzielone na moduły, jednak z perspektywy wdrożeniowej i uruchomieniowej stanowią jeden artefakt aplikacyjny.

Model monolityczny przez wiele lat dominował w projektowaniu aplikacji biznesowych, ponieważ pozwalał na względnie szybkie tworzenie i wdrażanie systemów o małej lub średniej skali. Jednolita struktura aplikacji upraszcza zarządzanie środowiskiem uruchomieniowym, proces wdrażania oraz kontrolę nad przepływem danych wewnątrz systemu. W takim podejściu komunikacja pomiędzy poszczególnymi modułami odbywa się zwykle w obrębie jednego procesu, co ogranicza narzut komunikacyjny i może pozytywnie wpływać na opóźnienia oraz wydajność.

2.1.1. *Cechy charakterystyczne architektury monolitycznej*

Podstawową cechą architektury monolitycznej jest zwarta struktura aplikacji. Oznacza to, że wszystkie kluczowe funkcje systemu są rozwijane, testowane i wdrażane jako jedna całość. Taka organizacja powoduje, że aplikacja zwykle korzysta z jednego procesu uruchomieniowego, jednego mechanizmu wdrażania oraz często jednej wspólnej bazy danych.

Architektura monolityczna charakteryzuje się również silnym powiązaniem poszczególnych komponentów. Z jednej strony ułatwia to wywoływanie funkcji pomiędzy modułami i ogranicza potrzebę stosowania rozbudowanych mechanizmów komunikacji sieciowej. Z drugiej strony zwiększa zależności pomiędzy elementami systemu, co może utrudniać niezależny rozwój, testowanie oraz skalowanie wybranych części aplikacji.

W praktyce oznacza to, że nawet niewielka zmiana w jednym fragmencie systemu może wymagać ponownego zbudowania i wdrożenia całej aplikacji. W przypadku małych projektów nie stanowi to zwykle istotnego problemu, jednak wraz ze wzrostem złożoności systemu może prowadzić do ograniczenia elastyczności rozwoju oraz wzrostu ryzyka błędów wdrożeniowych.

2.1.2. *Zalety i wady architektury monolitycznej*

Do najważniejszych zalet architektury monolitycznej należy prostota implementacyjna i operacyjna. Jednolity sposób budowania oraz wdrażania aplikacji upraszcza konfigurację środowiska, monitorowanie działania systemu oraz zarządzanie artefaktami wdrożeniowymi. Istotną zaletą jest również niska złożoność komunikacyjna, ponieważ wywołania pomiędzy

modułami realizowane są w ramach jednego procesu, bez konieczności używania protokołów sieciowych.

Architektura monolityczna może być także korzystna z punktu widzenia wydajności dla aplikacji o umiarkowanej skali, zwłaszcza wtedy, gdy głównym wymaganiem jest niski czas odpowiedzi i ograniczenie narzutu infrastrukturalnego. Mniejsza liczba komponentów uruchomieniowych oznacza również prostsze zarządzanie kosztami środowiska.

Jednocześnie rozwiązanie to posiada istotne ograniczenia. Wraz ze wzrostem rozmiaru systemu monolit staje się trudniejszy w rozwijaniu i utrzymaniu, ponieważ silne zależności pomiędzy modułami utrudniają izolowanie zmian. Problemem może być także skalowanie, które zwykle obejmuje całą aplikację, nawet jeśli zwiększone obciążenie dotyczy tylko jednej jej części. Prowadzi to do nieefektywnego wykorzystania zasobów obliczeniowych i może zwiększać koszty działania systemu w środowisku chmurowym.

2.2. Architektura mikroserwisowa

Architektura mikroserwisowa stanowi odmienne podejście do projektowania systemów informatycznych, w którym aplikacja dzielona jest na zestaw mniejszych, względnie niezależnych usług. Każda z nich realizuje określoną odpowiedzialność biznesową i może być rozwijana, wdrażana oraz skalowana niezależnie od pozostałych komponentów systemu.

Podejście mikroserwisowe zyskało dużą popularność wraz z rozwojem systemów chmurowych, konteneryzacji oraz narzędzi orkiestracyjnych. Jego główną ideą jest zwiększenie elastyczności architektury i umożliwienie lepszego dopasowania struktury systemu do złożonych procesów biznesowych. Dzięki temu możliwe staje się niezależne zarządzanie cyklem życia poszczególnych usług, co może przyspieszać rozwój systemu oraz ułatwiać pracę zespołów projektowych.

2.2.1. Cechy charakterystyczne architektury mikroserwisowej

Najważniejszą cechą architektury mikroserwisowej jest dekompozycja systemu na autonomiczne usługi. Każda usługa realizuje wyodrębnioną funkcję i komunikuje się z innymi usługami za pomocą zdefiniowanych interfejsów. W praktyce oznacza to, że logika aplikacji nie jest już wykonywana w jednym procesie, lecz rozproszona pomiędzy wiele niezależnych komponentów.

Mikroserwisy często posiadają własne zasoby danych lub przynajmniej własną warstwę odpowiedzialności za przetwarzanie określonego fragmentu informacji. Takie podejście zwiększa niezależność usług, ale jednocześnie komplikuje spójność danych, monitorowanie oraz obsługę błędów.

Istotnym elementem architektury mikroserwisowej jest także wysoki poziom rozproszenia. Wymusza on stosowanie mechanizmów komunikacji między usługami, rejestrowania instancji, równoważenia obciążenia, centralnego logowania, monitorowania oraz obsługi awarii. Z tego

względem mikroserwisów zapewniają większą elastyczność, ale kosztem większej złożoności systemowej i operacyjnej.

2.2.2. Zalety i wady architektury mikroserwisowej

Do najczęściej wskazywanych zalet architektury mikroserwisowej należy możliwość niezależnego rozwijania i wdrażania poszczególnych usług. Pozwala to ograniczyć wpływ zmian w jednym obszarze systemu na pozostałe komponenty oraz przyspiesza iteracyjny rozwój aplikacji. Istotną korzyścią jest również możliwość selektywnego skalowania tylko tych usług, które rzeczywiście stanowią wąskie gardło systemu.

Architektura mikroserwisowa dobrze współgra z środowiskami chmurowymi, ponieważ umożliwia wykorzystanie konteneryzacji, automatycznego skalowania oraz podejścia Infrastructure as Code. Dzięki temu może być szczególnie korzystna w dużych systemach o zmiennym obciążeniu i rozproszonych zespołach programistycznych.

Jednocześnie architektura ta posiada liczne wady. Rozproszenie usług powoduje wzrost narzutu komunikacyjnego, a komunikacja sieciowa pomiędzy komponentami generuje dodatkowe opóźnienia i zwiększa podatność systemu na awarie częściowe. Większa liczba elementów infrastruktury oznacza także bardziej złożone wdrażanie, monitorowanie i diagnozowanie problemów. W praktyce mikroserwisy mogą zatem prowadzić do wzrostu kosztów operacyjnych, zwłaszcza w przypadku systemów, dla których poziom złożoności biznesowej nie uzasadnia takiego modelu architektonicznego.

2.2.3. Metody komunikacji między mikroserwisami

Komunikacja pomiędzy mikroserwisami może być realizowana na różne sposoby, zależnie od wymagań funkcjonalnych i нефunkcjonalnych systemu. Najczęściej stosowane są dwa główne podejścia: komunikacja synchroniczna oraz asynchroniczna.

Komunikacja synchroniczna polega na bezpośrednim wywołaniu jednej usługi przez drugą, najczęściej przy użyciu protokołów takich jak HTTP/REST lub gRPC. Podejście to jest intuicyjne i stosunkowo proste w implementacji, jednak zwiększa zależność czasową pomiędzy usługami. Oznacza to, że niedostępność jednej usługi może bezpośrednio wpływać na działanie pozostałych komponentów systemu.

Komunikacja asynchroniczna opiera się zwykle na wykorzystaniu brokerów wiadomości i zdarzeń, takich jak kolejki komunikatów lub systemy publikacji-subskrypcji. Takie podejście zwiększa odporność systemu na chwilowe awarie i pozwala lepiej rozdzielić odpowiedzialność usług, lecz jednocześnie komplikuje śledzenie przepływu danych, debugowanie oraz zapewnienie spójności stanu.

Wybór mechanizmu komunikacji ma bezpośredni wpływ na wydajność, niezawodność i złożoność systemu. W systemach mikroserwisowych jest to jeden z kluczowych czynników wpływających na końcowe właściwości architektury.

2.3. Skalowalność i wydajność systemów rozproszonych

Wydajność systemu informatycznego jest zwykle rozumiana jako zdolność do sprawnego przetwarzania żądań przy zachowaniu akceptowalnych czasów odpowiedzi i odpowiedniego poziomu wykorzystania zasobów. W praktyce ocena wydajności obejmuje najczęściej takie metryki jak przepustowość, latencja, czas odpowiedzi, wykorzystanie procesora, pamięci operacyjnej oraz przepustowość sieci.

Skalowalność odnosi się natomiast do zdolności systemu do obsługi rosnącego obciążenia poprzez zwiększenie dostępnych zasobów. W środowiskach chmurowych wyróżnia się przede wszystkim dwa podstawowe modele skalowania: wertykalne i horyzontalne.

Skalowanie wertykalne polega na zwiększaniu zasobów pojedynczej instancji systemu, na przykład przez przydzielenie większej liczby rdzeni procesora lub większej ilości pamięci operacyjnej. Podejście to jest stosunkowo proste we wdrożeniu, lecz posiada naturalne ograniczenia wynikające z maksymalnych parametrów pojedynczej maszyny.

Skalowanie horyzontalne polega na zwiększaniu liczby instancji aplikacji lub jej poszczególnych komponentów. Jest ono szczególnie istotne w systemach rozproszonych i architekturze mikroserwisowej, gdzie wybrane usługi mogą być replikowane niezależnie od innych części systemu. Takie podejście pozwala lepiej dopasować zasoby do charakteru obciążenia, ale wymaga stosowania dodatkowych mechanizmów równoważenia obciążenia, koordynacji oraz monitorowania.

W kontekście porównania architektury monolitycznej i mikroserwisowej istotne jest, że oba style architektoniczne reagują na skalowanie w odmienny sposób. Monolit może osiągać dobre wyniki przy prostszej konfiguracji i mniejszym narzucie komunikacyjnym, natomiast mikroserwisy mogą oferować większą elastyczność skalowania, ale kosztem wzrostu złożoności i narzutu infrastrukturalnego.

2.4. Koszty utrzymania aplikacji w środowiskach chmurowych

Jednym z kluczowych aspektów oceny nowoczesnych architektur systemowych są koszty utrzymania aplikacji, szczególnie w środowiskach chmurowych, gdzie zasoby rozliczane są na podstawie rzeczywistego wykorzystania. Koszty te obejmują nie tylko zasoby obliczeniowe, takie jak procesor, pamięć i przestrzeń dyskowa, lecz także koszty transferu danych, równoważenia obciążenia, monitorowania, przechowywania logów oraz zarządzania infrastrukturą.

W przypadku architektury monolitycznej model kosztowy bywa prostszy, ponieważ aplikacja jest zwykle wdrażana jako mniejsza liczba instancji i wymaga mniej złożonego środowiska operacyjnego. Jednocześnie skalowanie całego monolitu może prowadzić do nadmiarowego przydzielania zasobów, gdy zwiększone obciążenie dotyczy jedynie wybranej części funkcjonalnej systemu.

Architektura mikroserwisowa umożliwia bardziej precyzyjne skalowanie poszczególnych usług, co w pewnych scenariuszach może prowadzić do efektywniejszego wykorzystania zasobów. Z drugiej strony większa liczba usług, kontenerów, komponentów sieciowych i mechanizmów zarządzających zwiększa koszty stałe oraz operacyjne. Oznacza to, że mikroserwisy nie zawsze są rozwiązaniem tańszym, nawet jeśli oferują większą elastyczność.

Analiza kosztów utrzymania powinna zatem uwzględniać zarówno koszty bezpośrednie infrastruktury, jak i koszty pośrednie wynikające ze złożoności utrzymania, wdrażania oraz monitorowania systemu. Właśnie dlatego porównanie architektury monolitycznej i mikroserwisowej wymaga jednoczesnej oceny wydajności, skalowalności oraz kosztów operacyjnych.

2.5. Podsumowanie

Architektura monolityczna oraz architektura mikroserwisowa reprezentują dwa odmienne podejścia do projektowania i wdrażania systemów informatycznych. Monolit charakteryzuje się prostszą strukturą, niższym narzutem komunikacyjnym oraz łatwiejszym zarządzaniem środowiskiem, jednak może napotykać ograniczenia w zakresie elastycznego rozwoju i skalowania. Mikroserwisy oferują większą modularność i możliwość niezależnego skalowania komponentów, ale jednocześnie zwiększają złożoność systemu oraz koszty operacyjne.

W środowiskach chmurowych różnice pomiędzy tymi podejściami stają się szczególnie istotne, ponieważ wpływają nie tylko na wydajność aplikacji, lecz także na sposób wykorzystania zasobów i całkowity koszt utrzymania systemu. Z tego względu porównanie obu architektur powinno być prowadzone w oparciu o mierzalne kryteria i kontrolowane warunki eksperymentalne. Stanowi to punkt wyjścia do dalszych rozważań przedstawionych w kolejnych rozdziałach pracy.

3. SYSTEMATYCZNY PRZEGLĄD LITERATURY

3.1. Cel i zakres przeglądu

Systematyczny przegląd literatury przeprowadzono w celu uporządkowania aktualnego stanu wiedzy dotyczącego porównania architektury monolitycznej i mikroserwisowej w środowiskach chmurowych. Przegląd miał na celu identyfikację publikacji naukowych analizujących wpływ stylu architektonicznego na wydajność, skalowalność oraz koszty utrzymania systemów. Istotnym założeniem było również rozpoznanie czynników kontekstowych, takich jak konteneryzacja, orkiestracja, obserwowalność czy sposób komunikacji między usługami, które mogą modyfikować wyniki porównań.

Przegląd literatury pełni w niniejszej pracy podwójną funkcję. Po pierwsze, pozwala określić, jakie podejścia badawcze dominują w dotychczasowych opracowaniach i jakie metryki są najczęściej wykorzystywane do oceny obu architektur. Po drugie, umożliwia wskazanie luk badawczych, które uzasadniają przeprowadzenie własnego eksperymentu porównawczego na funkcjonalnie równoważnych implementacjach tej samej aplikacji.

3.2. Pytania badawcze

Na potrzeby przeglądu sformułowano główne pytanie badawcze:

RQ: Jakie są różnice w wydajności, skalowalności i kosztach utrzymania pomiędzy architekturą monolityczną a mikroserwisową w środowiskach chmurowych oraz jakie czynniki kontekstowe najbardziej wpływają na obserwowane wyniki?

W celu uporządkowania analizy główne pytanie badawcze rozbito na cztery pytania szczegółowe:

- **RQ1:** Jak architektura monolityczna i mikroserwisowa wypadają w porównaniu pod względem wydajności, mierzonej za pomocą takich wskaźników jak czas odpowiedzi, latencja, przepustowość oraz dostępność?
- **RQ2:** Jakie różnice występują pomiędzy obiema architekturami w zakresie skalowalności pionowej i poziomej oraz zachowania systemu przy rosnącym obciążeniu?
- **RQ3:** Jakie koszty utrzymania i koszty operacyjne są raportowane dla architektury monolitycznej i mikroserwisowej, w szczególności w odniesieniu do infrastruktury, utrzymania, obserwowalności oraz procesów DevOps?
- **RQ4:** W jakich warunkach technologicznych i środowiskowych, takich jak rodzaj aplikacji, użyty stos technologiczny, środowisko chmurowe czy wykorzystanie kontenerów i orkiestracji, wyniki porównań ulegają zmianie?

3.3. Metodyka przeglądu

Przegląd przeprowadzono zgodnie z podejściem systematycznym, obejmującym zdefiniowanie pytania badawczego, opracowanie strategii wyszukiwania, wybór źródeł, zastosowanie kryteriów włączenia i wykluczenia, a następnie ekstrakcję oraz syntezę danych. Jako ramę konceptualną przyjęto model PICO, który dobrze odpowiada charakterowi porównawczemu niniejszego badania:

- **P (Population/Problem):** systemy i aplikacje wdrażane w środowiskach chmurowych lub cloud-native,
- **I (Intervention):** architektura mikroservisowa,
- **C (Comparison):** architektura monolityczna,
- **O (Outcome):** wydajność, skalowalność, koszty utrzymania, operacyjność i utrzymywalność.

Takie ujęcie umożliwiło zachowanie spójności pomiędzy strategią wyszukiwania a późniejszą syntezą wyników.

3.4. Strategia wyszukiwania

Strategia wyszukiwania została oparta na czterech grupach pojęć logicznie związanych z tematem pracy: (1) style architektoniczne, (2) środowiska chmurowe i uruchomieniowe, (3) metryki techniczne oraz (4) koszty i utrzymanie. Wewnątrz każdej grupy stosowano operator logiczny OR, natomiast pomiędzy grupami operator AND. Dzięki temu możliwe było jednoczesne zawężenie wyników do publikacji rzeczywiście odnoszących się do porównania monolitu i mikroservisów w kontekście chmurowym.

W praktyce wykorzystano następujące grupy słów kluczowych:

- **Architektura:** microservice*, "micro-service*", "microservices architecture", "microservice architecture", "service-oriented architecture", monolith*, "monolithic architecture", "monolithic application",
- **Chmura i runtime:** cloud, "cloud computing", "public cloud", "cloud-native", kubernetes, container*, docker, orchestration,
- **Metryki techniczne:** performance, benchmark*, throughput, latency, "response time", scalab*, autoscal*, elasticity,
- **Koszty i utrzymanie:** cost, "operational cost", "infrastructure cost", TCO, "total cost of ownership", maintenance, maintainability, reliability, availability, observability, DevOps.

Główne zapytanie wyszukiwawcze zastosowane w bazie Scopus miało następującą postać:

TITLE-ABS-KEY (

(


```

        microservice* OR "micro-service*" OR "microservices architecture"
        OR "microservice architecture" OR "service-oriented architecture"
    )
    AND
    (
        monolith* OR "monolithic architecture" OR "monolithic application"
        OR monolith
    )
    AND
    (
        cloud OR "cloud computing" OR "public cloud" OR "cloud-native"
        OR kubernetes OR container* OR docker OR orchestration
    )
    AND
    (
        performance OR benchmark* OR throughput OR latency
        OR "response time" OR scalab* OR autoscal* OR elasticity
    )
    AND
    (
        cost OR "operational cost" OR "infrastructure cost" OR TCO
        OR "total cost of ownership" OR maintenance OR maintainability
        OR reliability OR availability OR observability OR DevOps
    )
)
AND PUBYEAR > 2016
AND ( LIMIT-TO ( DOCTYPE , "ar" ) OR LIMIT-TO ( DOCTYPE , "cp" ) )
AND ( LIMIT-TO ( LANGUAGE , "English" ) )

```

Ze względu na różnice składniowe pomiędzy wyszukiwarkami poszczególnych baz zapytanie było adaptowane lokalnie, przy zachowaniu niezmienionej logiki semantycznej. Krótsze warianty zapytań stosowano między innymi w IEEE Xplore i ACM Digital Library, gdzie zbyt rozbudowane formuły powodowały nadmiar wyników ubocznych lub ograniczenia wyszukiwarki.

3.5. Źródła danych i zakres przeszukiwania

Wyszukiwanie literatury przeprowadzono w bazach uznawanych za podstawowe źródła publikacji z obszaru informatyki i inżynierii oprogramowania. W szczególności uwzględniono Scopus, IEEE Xplore, ACM Digital Library, SpringerLink oraz ScienceDirect. Dobór źródeł wynikał z potrzeby połączenia szerokiego pokrycia literatury interdyscyplinarnej z dobrym pokryciem publikacji stricte informatycznych i konferencyjnych.

Uwzględniono publikacje opublikowane w języku angielskim po roku 2016. Przyjęcie takiego zakresu czasowego było uzasadnione faktem, że po 2016 roku nastąpił wyraźny

wzrost liczby badań dotyczących systemów mikroserwisowych, konteneryzacji i cloud-native. Wyszukiwanie obejmowało artykuły naukowe oraz materiały konferencyjne, natomiast pominięto publikacje o charakterze czysto marketingowym, notki redakcyjne oraz opracowania bez wartości empirycznej.

3.6. Kryteria włączenia i wyłączenia

W celu zapewnienia spójności procesu selekcji zdefiniowano kryteria włączenia i wyłączenia.

3.6.1. Kryteria włączenia

Do dalszej analizy kwalifikowano publikacje, które spełniały co najmniej jedno z poniższych kryteriów:

- porównywały architekturę mikroserwisową i monolityczną bezpośrednio lub zawierały dane umożliwiające takie porównanie,
- dotyczyły systemów uruchamianych w środowiskach chmurowych, kontenerowych lub cloud-native,
- raportowały wyniki odnoszące się do wydajności, skalowalności, utrzymywalności, kosztów lub operacyjności,
- miały charakter empiryczny, eksperymentalny, benchmarkingowy lub zawierały systematyczną analizę literatury istotną dla przyjętego problemu badawczego.

3.6.2. Kryteria wyłączenia

Z analizy wyłączano publikacje, które:

- nie dotyczyły bezpośrednio architektury monolitycznej i mikroserwisowej,
- nie odnosiły się do środowisk chmurowych, kontenerowych ani nowoczesnych środowisk wdrożeniowych,
- miały charakter opiniotwórczy, marketingowy lub przeglądowy bez użytecznych danych analitycznych,
- koncentrowały się wyłącznie na porównaniu mikroserwisów z innymi stylami architektonicznymi, na przykład SOA, bez odniesienia do monolitu,
- były duplikatami rekordów pozyskanych z innych baz.

3.7. Proces selekcji publikacji

Proces selekcji miał charakter dwuetapowy. W pierwszym etapie przeprowadzono screening tytułów i abstraktów, odrzucając publikacje jednoznacznie niezwiązane z problemem badawczym. W drugim etapie analizowano pełne teksty prac zakwalifikowanych wstępnie, sprawdzając ich zgodność z pytaniami badawczymi oraz przyjętymi kryteriami selekcji.

Do organizacji procesu screeningu i usuwania duplikatów wykorzystano narzędzie Rayyan, które umożliwia sprawne zarządzanie rekordami bibliograficznymi, ich tagowanie oraz wspomaga deduplikację. Dodatkowo publikacjom przypisywano znaczniki tematyczne odpowiadające głównym osiom analizy, takim jak *performance*, *scalability*, *cost*, *Kubernetes*, *benchmark* czy *empirical study*. Takie podejście ułatwiło późniejszą syntezę wyników.

Po zakończeniu procesu deduplikacji i selekcji końcowy zbiór obejmował 16 publikacji. Rozkład publikacji według roku wydania wskazuje, że największa liczba prac została opublikowana w 2023 roku, natomiast wyraźny wzrost zainteresowania tematyką jest widoczny od 2020 roku. Na wykresie przedstawiającym rozkład czasowy widać odpowiednio: 1 publikację z 2017 roku, brak publikacji z lat 2018–2019, 2 publikacje z 2020 roku, 2 z 2021 roku, 3 z 2022 roku, 4 z 2023 roku oraz po 2 publikacje z lat 2024 i 2025. [slr_year_chart]

Znacząca część końcowego zbioru pochodziła z IEEE Xplore, które stanowiło 56,25% wszystkich zakwalifikowanych publikacji. Drugim najważniejszym źródłem był SpringerLink z udziałem 18,75%, natomiast ACM Digital Library, ScienceDirect, MDPI oraz CEUR-WS odpowiadały po 6,25% zbioru. Taki rozkład potwierdza, że rdzeń analizowanej literatury ma charakter informatyczny i konferencyjny, z silną reprezentacją publikacji indeksowanych w źródłach technicznych. [slr_sources_chart]

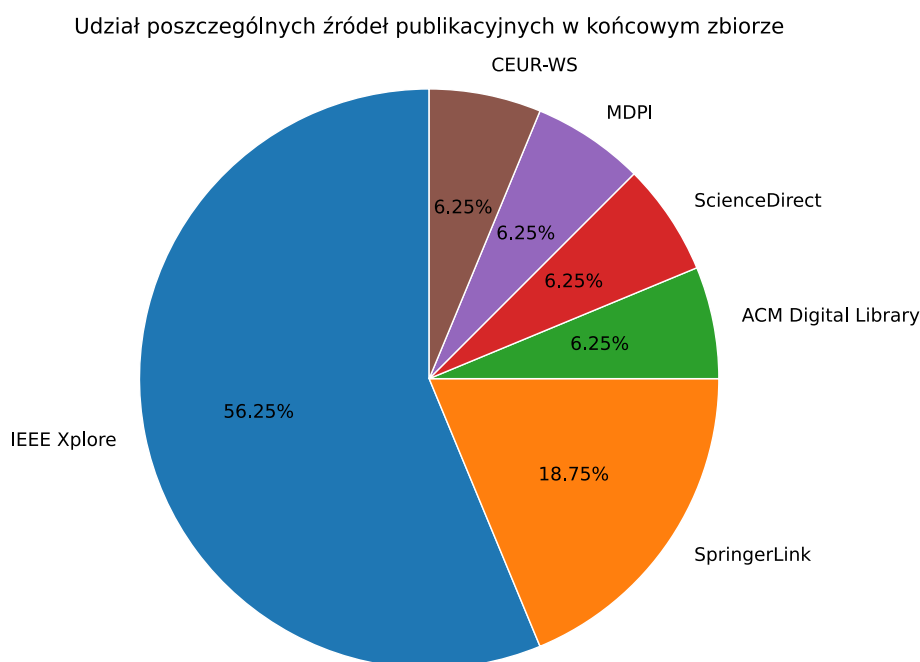


Rys. 3.1. Liczba wybranych publikacji w zależności od roku wydania

3.8. Ekstrakcja i synteza danych

Dla każdej zakwalifikowanej publikacji przeprowadzono ekstrakcję danych według jednolitego schematu. Rejestrowano przede wszystkim:

- typ badania,
- analizowany styl architektoniczny,
- kontekst wdrożeniowy,
- zastosowane technologie,
- metryki oceny,
- główne wyniki oraz ograniczenia badania.



Rys. 3.2. Udział poszczególnych źródeł publikacyjnych w końcowym zbiorze

W celu uporządkowania materiału źródłowego zastosowano syntezę tematyczną. Publikacje podzielono na trzy główne grupy odpowiadające celom niniejszej pracy:

1. badania koncentrujące się na wydajności,
2. badania dotyczące skalowalności i zachowania pod obciążeniem,
3. badania odnoszące się do kosztów utrzymania, utrzymywalności oraz złożoności operacyjnej.

Taki podział umożliwił przejrzyste zestawienie wyników literaturowych z rezultatami własnego eksperymentu zaprezentowanego w dalszych rozdziałach pracy.

3.9. Analiza badań dotyczących porównania architektury monolitycznej i mikroserwisowej

3.9.1. Porównania pod względem wydajności

W obszarze wydajności literatura najczęściej wykorzystuje takie metryki jak czas odpowiedzi, latencja, przepustowość oraz zużycie zasobów obliczeniowych. W wielu opracowaniach wskazuje się, że architektura monolityczna osiąga korzystniejsze wyniki w scenariuszach, w których logika aplikacji wykonywana jest w obrębie jednego procesu, bez konieczności wielokrotnej komunikacji sieciowej pomiędzy usługami. Brak kosztów serializacji, deserializacji oraz zdalnych wywołań procedur ogranicza narzut komunikacyjny i może skutkować niższą latencją.

Z drugiej strony literatura wskazuje, że architektura mikroserwisowa może osiągać porównywalną lub wyższą efektywność w tych scenariuszach, w których możliwe jest

odseparowanie najbardziej obciążonych funkcji i niezależna optymalizacja krytycznych komponentów. W takim ujęciu sama architektura nie determinuje jeszcze wyniku; istotne znaczenie mają również sposób dekompozycji systemu, rodzaj protokołu komunikacyjnego, liczba zależności pomiędzy usługami oraz charakter obciążenia.

3.9.2. Porównania pod względem skalowalności

W zakresie skalowalności literatura konsekwentnie akcentuje przewagę mikroserwisów wynikającą z możliwości niezależnego skalowania poszczególnych komponentów. W środowiskach chmurowych i kontenerowych szczególnie istotne okazuje się to, że tylko wybrane usługi mogą być skalowane poziomo w odpowiedzi na wzrost obciążenia, bez konieczności replikowania całej aplikacji.

Jednocześnie część badań zwraca uwagę, że korzyści te nie są darmowe. Wraz ze wzrostem liczby usług wzrasta złożoność zarządzania komunikacją, routingiem, monitorowaniem i równoważeniem obciążenia. Oznacza to, że przewaga architektury mikroserwisowej w zakresie elastyczności może zostać częściowo osłabiona przez narzut operacyjny, szczególnie w systemach niewielkich lub średnio złożonych.

3.9.3. Porównania pod względem kosztów utrzymania

Najmniej jednoznaczne wyniki dotyczą kosztów utrzymania. W części publikacji koszty są rozumiane wąsko, jako koszt infrastrukturalny związany z użyciem procesora, pamięci, maszyn wirtualnych lub kontenerów. W innych badaniach stosuje się ujęcie szersze, obejmujące również koszty obserwowalności, automatyzacji wdrożeń, utrzymania pipeline'ów CI/CD, zarządzania konfiguracją oraz koordynacji zespołów.

Z literatury wynika, że architektura monolityczna bywa korzystniejsza kosztowo na wczesnych etapach rozwoju systemu lub w projektach o umiarkowanej złożoności. Mikroserwisy mogą z kolei uzasadniać większy koszt operacyjny wtedy, gdy skala systemu, zmienność obciążenia lub organizacja pracy zespołów pozwala realnie wykorzystać ich modularność i niezależne skalowanie.

3.10. Luki badawcze i uzasadnienie własnego podejścia

Analiza wybranych publikacji wskazuje na kilka istotnych luk badawczych. Po pierwsze, stosunkowo niewiele prac porównuje architekturę monolityczną i mikroserwisową na podstawie funkcjonalnie równoważnych implementacji tej samej aplikacji. W wielu przypadkach porównywane systemy różnią się nie tylko stylem architektonicznym, lecz także zakresem funkcjonalnym lub zastosowanym stosem technologicznym, co utrudnia wyciąganie jednoznacznych wniosków.

Po drugie, duża część badań koncentruje się wyłącznie na jednym wymiarze oceny, najczęściej wydajności albo skalowalności, pomijając koszty utrzymania. Tymczasem w praktyce

inżynierskiej decyzja architektoniczna ma charakter wielokryterialny i powinna uwzględniać także nakład operacyjny oraz koszty środowiska uruchomieniowego.

Po trzecie, literatura rzadko oferuje jednolitą metodykę eksperymentalną pozwalającą na bezpośrednie porównywanie wyników. Różnice dotyczą zarówno metryk, jak i sposobu wdrożenia, typu infrastruktury, poziomu obciążenia czy użytych narzędzi. W efekcie nadal brakuje badań, które w jednym, spójnym eksperymencie zestawiałyby wydajność, skalowalność i koszt utrzymania obu architektur w kontrolowanych warunkach.

Zidentyfikowane luki uzasadniają podejście przyjęte w niniejszej pracy. Polega ono na porównaniu funkcjonalnie równoważnych wariantów tej samej aplikacji, uruchomionych w porównywalnym środowisku chmurowym i badanych według jednolitej procedury eksperymentalnej. Takie podejście pozwala ograniczyć wpływ czynników zakłócających i lepiej odpowiedzieć na pytanie o rzeczywisty wpływ stylu architektonicznego na parametry jakościowe systemu.

3.11. Podsumowanie

Przeprowadzony systematyczny przegląd literatury potwierdził, że porównanie architektury monolitycznej i mikroserwisowej pozostaje istotnym i aktualnym problemem badawczym. Jednocześnie analiza wykazała, że dominują opracowania koncentrujące się na pojedynczych aspektach jakościowych, podczas gdy liczba badań obejmujących równocześnie wydajność, skalowalność i koszty utrzymania jest wciąż ograniczona.

Wnioski z przeglądu wskazują, że architektura monolityczna może oferować niższy narzut komunikacyjny i większą prostotę operacyjną, natomiast architektura mikroserwisowa zapewnia większą elastyczność skalowania i modularność. Ostateczna ocena obu podejść zależy jednak od kontekstu wdrożeniowego, charakterystyki aplikacji oraz sposobu implementacji środowiska uruchomieniowego.

Uzyskane wyniki przeglądu stanowią podstawę do zaprojektowania własnego eksperymentu badawczego, przedstawionego w kolejnych rozdziałach pracy.

4. IMPLEMENTACJA APLIKACJI BENCHMARKOWEJ

4.1. Cel rozdziału

Celem niniejszego rozdziału jest przedstawienie implementacji aplikacji benchmarkowej wykorzystanej w badaniach porównawczych. Aplikację przygotowano w dwóch wariantach architektonicznych: jako system monolityczny oraz jako system oparty na mikroservisach. Oba warianty realizują ten sam proces biznesowy i operują na tych samych danych wejściowych oraz wyjściowych. Dzięki temu możliwe było przeprowadzenie rzetelnego porównania wpływu architektury na wydajność, skalowalność oraz koszty utrzymania.

W celu zwiększenia wartości porównawczej eksperymentu każdą z wersji architektonicznych zaimplementowano w trzech stosach technologicznych:

- Java z wykorzystaniem frameworka Spring,
- JavaScript z wykorzystaniem środowiska Node.js,
- Python z wykorzystaniem frameworka Flask.

W rezultacie przygotowano sześć implementacji aplikacji: trzy monolity oraz trzy warianty mikroservisowe. Pozwala to analizować nie tylko wpływ samego stylu architektonicznego, ale również sprawdzić, czy obserwowane zależności są spójne pomiędzy różnymi ekosystemami programistycznymi.

4.2. Charakterystyka aplikacji benchmarkowej

Aplikacja benchmarkowa została osadzona w domenie *image processing*. Jej zadaniem jest wyszukiwanie obrazów podobnych do obrazu referencyjnego na podstawie ekstrakcji cech oraz porównania z wcześniej przygotowanym indeksem wektorów cech. Wynik działania systemu stanowi lista obrazów spełniających zadany próg podobieństwa wraz z odpowiadającymi im metadanymi.

Wybrany scenariusz jest odpowiedni do badań architektonicznych z kilku powodów. Po pierwsze, obejmuje kilka kolejnych etapów przetwarzania, które mogą zostać naturalnie wydzielone do osobnych komponentów. Po drugie, zawiera zarówno element obliczeniowy, związany z ekstrakcją cech z obrazu, jak i etap wyszukiwania podobieństwa oraz pobierania metadanych. Po trzecie, logika ta może zostać zrealizowana zarówno w jednej aplikacji monolitycznej, jak i w formie rozproszonego łańcucha mikroservisów, bez zmiany sensu biznesowego całego systemu.

W obu wariantach architektonicznych aplikacja korzysta z tego samego zbioru danych wejściowych, obejmującego:

- buforowane obrazy referencyjne,
- indeks cech obrazów zapisany w pliku `features.json`,
- metadane obrazów zapisane w pliku `metadata.json`.

Dane te są ładowane przy starcie aplikacji do struktur pamięciowych, co pozwala ograniczyć wpływ zewnętrznej warstwy persystencji na wyniki eksperymentu. Dzięki temu badanie koncentruje się na wpływie architektury systemu i modelu komunikacji, a nie na kosztach dostępu do bazy danych.

4.3. Model wejścia i wyjścia systemu

Z perspektywy klienta testowego aplikacja udostępnia interfejs HTTP. W przedstawionym scenariuszu badawczym punkt końcowy przyjmuje parametry sterujące procesem wyszukiwania obrazów podobnych. Najważniejszymi parametrami są próg podobieństwa oraz rozmiar jądra wykorzystywanego w etapie ekstrakcji cech.

Przykładowe wywołanie punktu końcowego można przedstawić następująco:

Listing 4.1: Przykładowe wywołanie punktu końcowego HTTP

```
GET /images/search?threshold=0.2&kernel=3
```

Logiczny model żądania można również zapisać w postaci uproszczonego obiektu JSON:

Listing 4.2: Logiczny model żądania wyszukiwania

```
{
  "threshold": 0.2,
  "kernel": 3
}
```

W odpowiedzi system zwraca listę obiektów reprezentujących obrazy uznane za podobne do obrazu referencyjnego. Każdy element odpowiedzi zawiera identyfikator obrazu, jego nazwę oraz listę tagów opisujących zawartość. Przykładową odpowiedź przedstawiono w listingu 4.3.

Listing 4.3: Przykładowa odpowiedź systemu

```
[
  {
    "id": "img_001",
    "name": "forest-path",
    "tags": ["forest", "nature", "path"]
  },
  {
    "id": "img_014",
    "name": "green-woods",
    "tags": ["trees", "forest", "landscape"]
  }
]
```


Z punktu widzenia eksperymentu istotne jest to, że wszystkie implementacje zwracają ten sam typ odpowiedzi końcowej. Pozwala to uznać badane warianty za funkcjonalnie równoważne.

4.4. Logika przetwarzania obrazu

Proces biznesowy realizowany przez aplikację składa się z kilku kolejnych etapów. Najpierw wybierany jest obraz referencyjny, który w wariacie testowym pobierany jest z wcześniej załadowanego zbioru obrazów. Następnie z obrazu wyznaczany jest wektor cech. W przedstawionej implementacji cechy są obliczane na podstawie uproszczonego przetwarzania obrazu, obejmującego:

- konwersję bufora obrazu do tablicy wartości pikseli,
- normalizację do skali szarości,
- zastosowanie uśredniającego jądra konwolucyjnego o zadanym rozmiarze,
- budowę histogramu o ustalonej liczbie przedziałów,
- normalizację histogramu do postaci wektora cech.

Otrzymany wektor porównywany jest następnie z indeksem cech wszystkich obrazów zapisanych w systemie. Miara podobieństwa została oparta na odległości euklidesowej pomiędzy wektorem zapytania a wektorami zapisanymi w indeksie. Do wyniku wyszukiwania trafiają identyfikatory tych obrazów, dla których odległość nie przekracza zadanego progu podobieństwa.

W ostatnim etapie system pobiera metadane dla znalezionych identyfikatorów obrazów i buduje odpowiedź zwracaną do klienta. Taki przepływ danych jest wspólny dla wariantu monolitycznego i mikroserwisowego. Różnica dotyczy wyłącznie sposobu rozmieszczenia odpowiedzialności pomiędzy komponentami oraz modelu komunikacji wewnętrznej.

4.5. Implementacja wariantu monolitycznego

Wariant monolityczny został zaimplementowany jako pojedyncza aplikacja uruchamiana w jednym procesie. W tej wersji wszystkie etapy przetwarzania obrazu, tj. obsługa żądania HTTP, ekstrakcja cech, wyszukiwanie podobieństwa oraz pobieranie metadanych, realizowane są lokalnie wewnątrz jednego artefaktu wdrożeniowego.

W implementacji Java wariant monolityczny został oparty na frameworku Spring. Aplikacja udostępnia punkt końcowy `/images/search`, a podczas uruchamiania ładuje obrazy, indeks cech oraz metadane do struktur pamięciowych. Przepływ wykonania obejmuje kolejno: odczyt parametrów wejściowych, wybór obrazu referencyjnego, ekstrakcję cech, porównanie z indeksem oraz mapowanie wyników do odpowiedzi zwracanej klientowi. Analogiczny model działania zastosowano również w implementacjach JavaScript i Python.

Takie podejście upraszcza strukturę systemu. Komunikacja pomiędzy poszczególnymi częściami logiki odbywa się przez lokalne wywołania funkcji lub metod, bez konieczności użycia protokołów sieciowych. Oznacza to niższy narzut komunikacyjny, prostszy model wdrożenia oraz

mniejszą liczbę komponentów operacyjnych. Z drugiej strony cały system stanowi jeden artefakt, który w przypadku skalowania poziomego musi być replikowany jako całość.

Schemat infrastruktury wariantu monolitycznego przedstawiono na rysunku 4.1, natomiast sekwencję realizacji żądania w tym wariantcie pokazano na rysunku 4.2.

4.6. Implementacja wariantu mikroserwisowego

Wariant mikroserwisowy został zbudowany jako system złożony z pięciu współpracujących usług. Każda z nich odpowiada za wyodrębniony etap przetwarzania, a komunikacja pomiędzy usługami realizowana jest za pomocą gRPC. Zastosowanie tego protokołu miało na celu ograniczenie narzutu komunikacyjnego względem klasycznego modelu HTTP/JSON oraz zapewnienie jednolitego kontraktu między komponentami systemu.

Architektura mikroserwisowa obejmuje następujące komponenty:

- **Gateway** – warstwa wejściowa udostępniająca interfejs HTTP dla klienta zewnętrznego,
- **Search** – usługa orkiestrująca cały proces wyszukiwania,
- **Feature** – usługa odpowiedzialna za ekstrakcję wektora cech z obrazu referencyjnego,
- **Similarity** – usługa porównująca wektor zapytania z indeksem cech i zwracająca listę identyfikatorów obrazów spełniających próg podobieństwa,
- **Metadata** – usługa pobierająca metadane obrazów dla identyfikatorów zwróconych przez usługę *Similarity*.

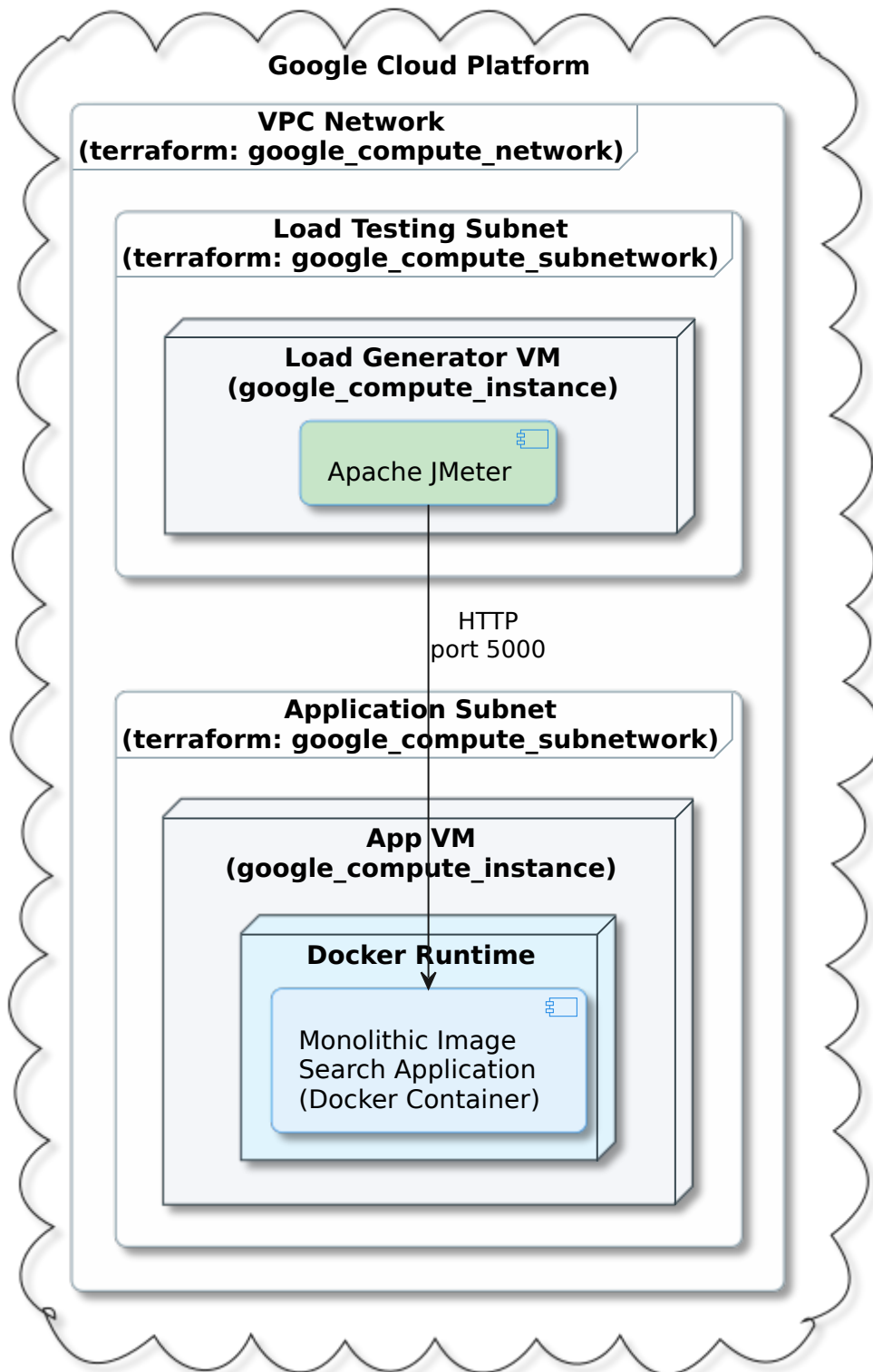
Przepływ przetwarzania w tym wariantcie przebiega następująco:

1. klient wysyła żądanie HTTP do usługi *Gateway*,
2. *Gateway* przekazuje żądanie do usługi *Search* przy użyciu gRPC,
3. *Search* wywołuje usługę *Feature*, która oblicza wektor cech obrazu referencyjnego,
4. *Search* przekazuje otrzymany wektor do usługi *Similarity*,
5. *Similarity* porównuje wektor z indeksem i zwraca listę identyfikatorów obrazów,
6. *Search* przekazuje identyfikatory do *Gateway*,
7. *Gateway* wywołuje usługę *Metadata*, która pobiera metadane dla wskazanych obrazów,
8. *Gateway* buduje końcową odpowiedź JSON i zwraca ją klientowi.

Takie rozdzielenie odpowiedzialności pozwala lepiej odwzorować charakter systemów rozproszonych oraz umożliwia niezależne skalowanie wybranych komponentów. Jednocześnie wprowadza dodatkowy koszt związany z komunikacją pomiędzy usługami, serializacją wiadomości, zarządzaniem większą liczbą procesów oraz większą złożonością wdrożeniową.

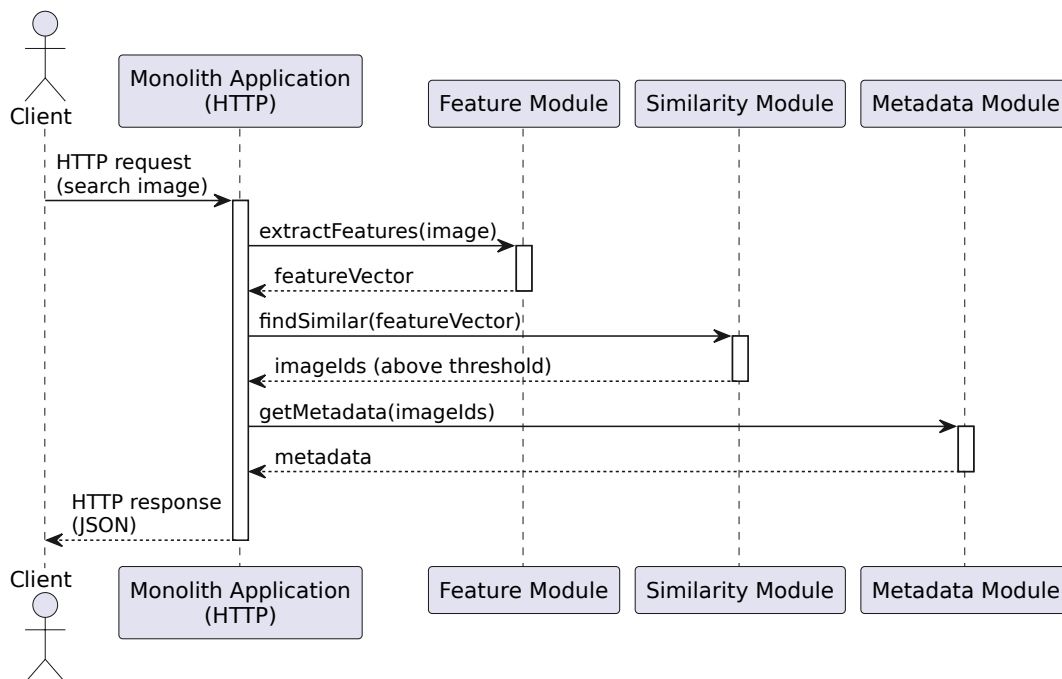
Schemat infrastruktury wariantu mikroserwisowego przedstawiono na rysunku 4.3, natomiast sekwencję przetwarzania w tym wariantcie pokazano na rysunku 4.4.

Deployment diagram - Benchmark environment (Monolith)



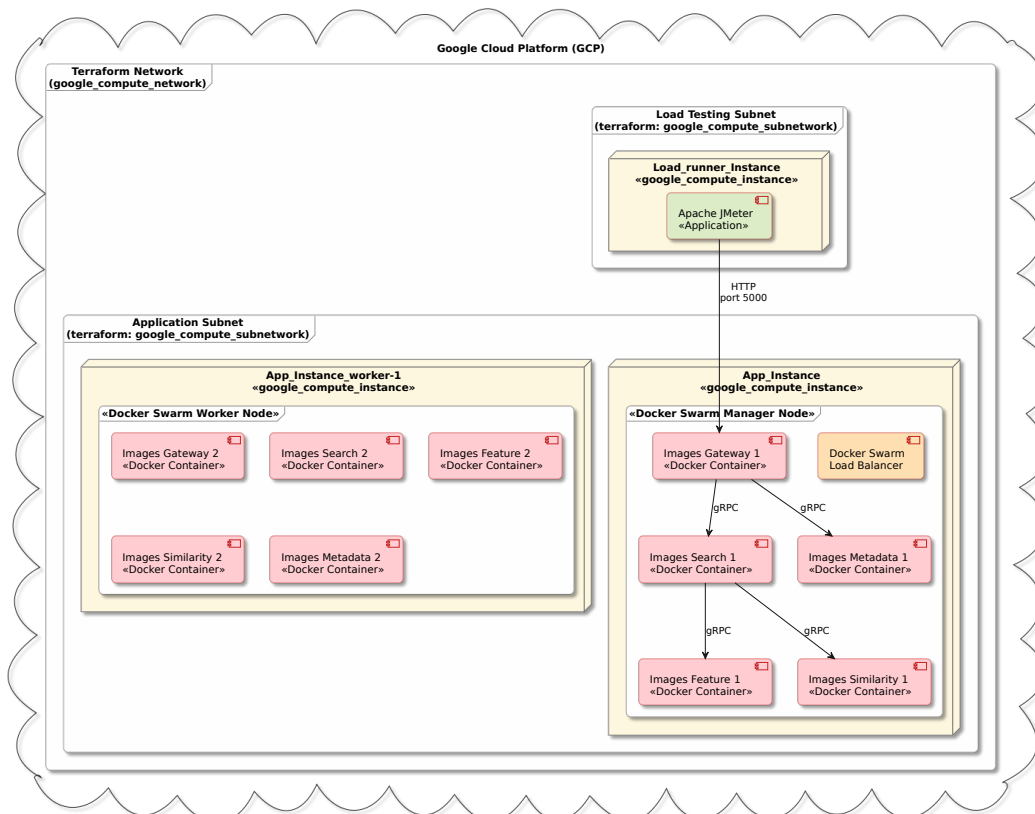
Rys. 4.1. Schemat infrastruktury wariantu monolitycznego

Diagram sekwencji - wyszukiwanie podobnych obrazów (monolit)

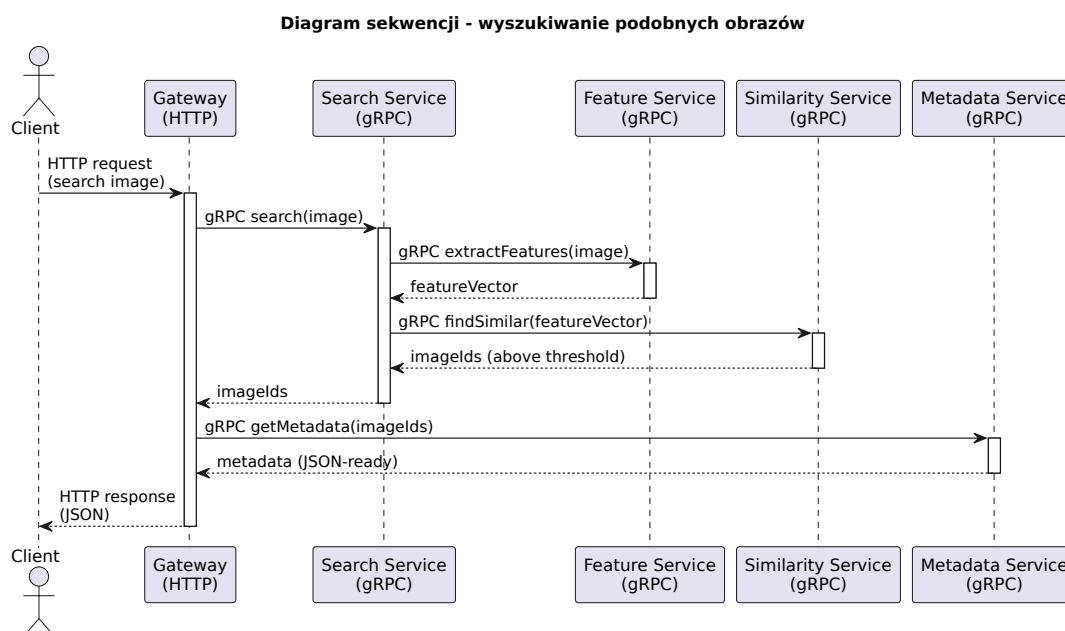


Rys. 4.2. Diagram sekwencji przetwarzania w wariacie monolitycznym

Deployment diagram - Benchmark environment (Images microservices)



Rys. 4.3. Schemat infrastruktury wariantu mikroserwisowego



Rys. 4.4. Diagram sekwencji przetwarzania w wariancie mikroserwisowym

4.7. Stosy technologiczne

Każdy z dwóch wariantów architektonicznych został zaimplementowany w trzech technologiach programistycznych. Takie podejście pozwala zmniejszyć ryzyko, że uzyskane wyniki będą specyficzne wyłącznie dla jednego środowiska uruchomieniowego.

4.7.1. Java i Spring

Implementacja w języku Java została oparta na frameworku Spring. Wariant monolityczny udostępnia REST-owy punkt końcowy wyszukiwania obrazów i realizuje wszystkie etapy przetwarzania w jednym procesie aplikacyjnym. Wariant mikroserwisowy wykorzystuje Spring jako bazę dla poszczególnych komponentów oraz biblioteki gRPC do komunikacji między usługami.

Java stanowi w badaniu reprezentanta środowiska o dojrzałym modelu wielowątkowości, rozbudowanym ekosystemie serwerowym oraz wysokiej przewidywalności zachowania w środowiskach produkcyjnych. Dzięki temu możliwe jest porównanie zachowania tej platformy z lżejszymi środowiskami interpretera i środowiska uruchomieniowego skryptowego.

4.7.2. JavaScript i Node.js

Wariant JavaScript został przygotowany w środowisku Node.js. Podobnie jak w przypadku Javy, przygotowano zarówno wersję monolityczną, jak i mikroserwisową z komunikacją gRPC. Zastosowanie Node.js pozwala uwzględnić w badaniu model wykonania oparty na pojedynczej pętli zdarzeń oraz nieblokującym przetwarzaniu operacji wejścia-wyjścia.

W kontekście badań porównawczych jest to istotne, ponieważ pozwala sprawdzić, jak architektura monolityczna i mikroserwisowa zachowują się w środowisku o odmiennym modelu współbieżności niż klasyczne środowisko JVM.

4.7.3. Python i Flask

Implementacja Python została oparta na frameworku Flask. Wariant monolityczny realizuje cały przepływ przetwarzania obrazu w jednej aplikacji, natomiast wariant mikroserwisowy rozdziela logikę pomiędzy pięć usług komunikujących się przez gRPC. Python włączono do badania jako reprezentanta środowiska szczególnie popularnego w zadaniach związanych z przetwarzaniem danych i obrazów.

Z punktu widzenia niniejszej pracy ważne było sprawdzenie, czy charakterystyka Pythona, w tym wyższy narzut interpretacyjny i odmienny model wykonania niż w Javie i Node.js, wpływa na skalę różnic pomiędzy monolitem a mikroserwisami.

4.8. Dane testowe i przygotowanie środowiska

We wszystkich implementacjach wykorzystano ten sam zbiór danych testowych. Obejmuje on:

- obrazy źródłowe przechowywane w katalogu `data/images`,
- wektory cech zapisane w pliku `data/features.json`,
- metadane obrazów zapisane w pliku `data/metadata.json`.

Podczas uruchamiania aplikacji dane te są ładowane do pamięci operacyjnej. Takie podejście pozwala ograniczyć wpływ opóźnień pochodzących od zewnętrznych systemów bazodanowych oraz zapewnia spójne warunki wejściowe dla wszystkich testowanych wariantów. W konsekwencji pomiar koncentruje się na wpływie architektury i technologii implementacyjnej, a nie na narzucie związanym z warstwą persystencji.

4.9. Znaczenie przyjętej implementacji dla badań eksperymentalnych

Przyjęta konstrukcja aplikacji benchmarkowej ma istotne znaczenie metodologiczne. Po pierwsze, oba warianty architektoniczne realizują dokładnie ten sam proces biznesowy, co ogranicza wpływ czynników funkcjonalnych na wyniki eksperymentu. Po drugie, zastosowanie trzech języków programowania pozwala zweryfikować, czy zależności obserwowane pomiędzy monolitem a mikroserwisami są spójne w różnych środowiskach wykonawczych. Po trzecie, rozdzielenie wariantu mikroserwisowego na pięć komponentów powoduje, że w systemie pojawia się rzeczywisty koszt komunikacji rozproszonej, a więc cecha kluczowa z punktu widzenia celu pracy.

Przygotowana aplikacja benchmarkowa umożliwia zatem analizę trzech kluczowych obszarów:

- wydajności, rozumianej jako czas odpowiedzi i przepustowość,
- skalowalności pionowej i poziomej,
- kosztów utrzymania wynikających z rozmiaru infrastruktury i liczby uruchamianych komponentów.

Tak zaprojektowane środowisko stanowi spójną i wiarygodną podstawę do dalszych badań eksperymentalnych przedstawionych w kolejnych rozdziałach pracy.

4.10. Podsumowanie

W rozdziale przedstawiono implementację aplikacji benchmarkowej wykorzystanej do porównania architektury monolitycznej i mikroserwisowej. Opisana aplikacja działa w domenie przetwarzania obrazów i realizuje proces wyszukiwania obrazów podobnych na podstawie ekstrakcji cech, porównania z indeksem oraz pobierania metadanych.

Aplikację zaimplementowano w trzech technologiach: Java z wykorzystaniem Spring, JavaScript w środowisku Node.js oraz Python z wykorzystaniem Flask. Dla każdej technologii przygotowano wersję monolityczną oraz wersję mikroserwisową opartą na pięciu usługach komunikujących się przez gRPC. Takie podejście zapewnia funkcjonalną równowagę badanych wariantów i tworzy właściwą podstawę do dalszej analizy wydajności, skalowalności i kosztów utrzymania.

5. METODYKA BADAŃ EKSPERYMENTALNYCH

5.1. Cele i scenariusze testowe

Celem badań eksperymentalnych jest porównanie dwóch wariantów architektonicznych tej samej aplikacji: wariantu monolitycznego oraz wariantu mikroserwisowego. Badanie zostało zaprojektowane w taki sposób, aby ocenić wpływ stylu architektonicznego na trzy główne obszary analizy: wydajność, skalowalność oraz koszty utrzymania w środowisku chmurowym.

W ramach eksperymentu przyjęto kilka scenariuszy testowych, różniących się konfiguracją środowiska oraz sposobem skalowania systemu. Pozwala to ocenić nie tylko zachowanie aplikacji przy standardowym obciążeniu, lecz także jej reakcję na wzrost zapotrzebowania na zasoby. Każdy scenariusz został przygotowany tak, aby umożliwić bezpośrednie porównanie wyników uzyskanych dla obu badanych architektur.

Scenariusze testowe obejmują trzy podstawowe grupy eksperymentów. Pierwsza z nich dotyczy testów bez zastosowania mechanizmów skalowania i pozwala ocenić bazową wydajność obu wariantów. Druga grupa obejmuje testy skalowania wertykalnego, w których zwiększane są zasoby pojedynczych instancji aplikacji. Trzecia grupa obejmuje testy skalowania horyzontalnego, polegające na zwiększaniu liczby instancji systemu lub jego wybranych komponentów.

Tak zdefiniowany układ eksperymentów umożliwia ocenę zachowania aplikacji w różnych warunkach pracy i stanowi podstawę do dalszej analizy porównawczej.

5.2. Narzędzia wykorzystane do pomiarów

Przeprowadzenie badań eksperymentalnych wymagało użycia zestawu narzędzi umożliwiających generowanie obciążenia, monitorowanie działania aplikacji, obserwację wykorzystania zasobów oraz gromadzenie wyników pomiarów. Zastosowane narzędzia dobrano w taki sposób, aby zapewnić powtarzalność testów oraz możliwość porównania wyników pomiędzy kolejnymi uruchomieniami eksperymentu.

Do generowania ruchu testowego wykorzystano narzędzie służące do przeprowadzania testów obciążeniowych, które umożliwia definiowanie scenariuszy żądań, kontrolę poziomu równoległości oraz pomiar podstawowych parametrów wydajnościowych, takich jak czas odpowiedzi i liczba obsłużonych żądań. Narzędzie to stanowi podstawę do oceny zachowania systemu pod rosnącym obciążeniem.

Do monitorowania środowiska wykorzystano mechanizmy pozwalające na obserwację parametrów infrastrukturalnych, takich jak zużycie procesora, pamięci operacyjnej, ruch sieciowy oraz liczba aktywnych instancji aplikacji. Dane te są niezbędne do oceny efektywności wykorzystania zasobów oraz do analizy wpływu skalowania na zachowanie systemu.

W badaniu wykorzystano również narzędzia wspierające automatyzację wdrożeń i wykonywania eksperymentów, co pozwoliło ograniczyć udział działań manualnych i zwiększyć porównywalność wyników. Dzięki temu każda seria testowa mogła zostać przeprowadzona według tej samej procedury.

5.3. Metryki oceny

Ocena badanych architektur została oparta na zestawie metryk pozwalających uchwycić zarówno wydajność aplikacji, jak i konsekwencje infrastrukturalne oraz kosztowe zastosowania danego stylu architektonicznego. Dobór metryk wynika bezpośrednio z celu pracy i podporządkowany jest trzem głównym obszarom porównania: wydajności, skalowalności oraz kosztom utrzymania.

5.3.1. Przepustowość

Przepustowość określa liczbę żądań, które system jest w stanie obsłużyć w jednostce czasu. Jest to jedna z podstawowych metryk oceny wydajności aplikacji, szczególnie istotna w testach obciążeniowych. W kontekście niniejszego badania przepustowość pozwala określić, który wariant architektoniczny efektywniej przetwarza rosnący strumień żądań.

Analiza przepustowości jest szczególnie ważna przy porównywaniu systemów wdrożonych w środowisku chmurowym, ponieważ pozwala ocenić, czy wzrost wykorzystania zasobów przekłada się na rzeczywisty wzrost zdolności przetwarzania. W badaniu przepustowość stanowi jedną z głównych metryk używanych do porównania architektury monolitycznej i mikroserwisowej.

5.3.2. Latencja

Latencja opisuje opóźnienie występujące pomiędzy wysłaniem żądania a otrzymaniem odpowiedzi przez klienta. W praktyce jest ona zwykle wyrażana jako czas odpowiedzi systemu i stanowi kluczową metrykę z punktu widzenia jakości odbieranej przez użytkownika końcowego.

W niniejszej pracy analiza latencji pozwala ocenić wpływ rozproszenia logiki biznesowej i komunikacji pomiędzy usługami na czas realizacji żądań. Ma to szczególne znaczenie przy porównywaniu systemu monolitycznego z wariantem mikroserwisowym, w którym dodatkowy narzut może wynikać z wywołań sieciowych, serializacji danych oraz pośrednictwa pomiędzy komponentami.

W badaniu analizowane są zarówno wartości średnie, jak i wybrane statystyki pozycyjne czasu odpowiedzi, co pozwala pełniej ocenić zachowanie aplikacji pod obciążeniem.

5.3.3. Zużycie zasobów obliczeniowych

Zużycie zasobów obliczeniowych obejmuje przede wszystkim wykorzystanie procesora, pamięci operacyjnej oraz, w razie potrzeby, ruchu sieciowego i innych parametrów

infrastrukturalnych. Metryka ta ma istotne znaczenie przy ocenie efektywności działania systemu, ponieważ pozwala określić, jakim kosztem infrastrukturalnym osiągane są określone wyniki wydajnościowe.

W kontekście porównania dwóch stylów architektonicznych analiza zasobów umożliwia ocenę, czy większa modularność i elastyczność mikroserwisów nie prowadzi jednocześnie do nadmiernego narzutu operacyjnego. W przypadku architektury monolitycznej metryka ta pozwala z kolei ocenić, czy prostszy model uruchomieniowy rzeczywiście przekłada się na bardziej ekonomiczne wykorzystanie infrastruktury.

5.3.4. Koszty utrzymania

Koszty utrzymania stanowią czwarty wymiar oceny badanych architektur i obejmują zarówno koszty bezpośrednie związane z wykorzystaniem zasobów chmurowych, jak i koszty pośrednie wynikające ze złożoności środowiska. W praktyce analiza ta uwzględnia przede wszystkim koszt instancji obliczeniowych, zasobów pamięciowych, mechanizmów równoważenia obciążenia, transferu danych oraz komponentów wspierających działanie systemu.

W przypadku niniejszego badania analiza kosztów nie ogranicza się wyłącznie do nominalnej ceny zasobów, lecz obejmuje także relację pomiędzy ponoszonym kosztem a uzyskiwaną wydajnością i skalowalnością. Taki sposób oceny pozwala lepiej określić rzeczywistą opłacalność zastosowania danego stylu architektonicznego.

5.4. Scenariusz testów bez skalowania

Pierwszy scenariusz eksperymentalny obejmuje testy przeprowadzane bez zastosowania dodatkowych mechanizmów skalowania. Celem tego etapu jest określenie bazowych właściwości obu wariantów aplikacji przy stałej konfiguracji środowiska uruchomieniowego.

W tym scenariuszu zarówno wariant monolityczny, jak i mikroserwisowy uruchamiane są w porównywalnej konfiguracji zasobów. Następnie do obu systemów kierowany jest ruch testowy o stopniowo rosnącym natężeniu, co pozwala zaobserwować momenty pogorszenia wydajności, wzrostu opóźnień oraz zmian w wykorzystaniu zasobów.

Scenariusz ten pełni rolę punktu odniesienia dla dalszych eksperymentów związanych ze skalowaniem i umożliwia ocenę, jak obie architektury zachowują się w warunkach bazowych.

5.5. Scenariusz testów skalowania wertykalnego

Drugi scenariusz obejmuje badanie wpływu skalowania wertykalnego na zachowanie systemu. Skalowanie wertykalne polega na zwiększaniu zasobów pojedynczej instancji aplikacji lub usługi, na przykład poprzez przydzielenie większej liczby rdzeni procesora lub większej ilości pamięci operacyjnej.

W ramach tego scenariusza wykonywane są kolejne serie testów dla różnych konfiguracji zasobów, przy zachowaniu tego samego schematu obciążenia. Pozwala to ocenić, w jakim stopniu zwiększenie parametrów pojedynczej instancji przekłada się na poprawę przepustowości, obniżenie latencji oraz zmianę kosztów utrzymania.

Scenariusz ten jest szczególnie istotny dla architektury monolitycznej, która często opiera się właśnie na skalowaniu pionowym, ale umożliwia również ocenę, w jakim stopniu taki mechanizm jest efektywny dla wybranych komponentów systemu mikroserwisowego.

5.6. Scenariusz testów skalowania horyzontalnego

Trzeci scenariusz dotyczy skalowania horyzontalnego, polegającego na zwiększaniu liczby instancji aplikacji lub wybranych usług. W środowiskach chmurowych jest to jeden z podstawowych mechanizmów zwiększania zdolności systemu do obsługi rosnącego obciążenia.

W przypadku wariantu monolitycznego skalowanie horyzontalne obejmuje replikację całej aplikacji, natomiast w wariantcie mikroserwisowym możliwe jest niezależne zwiększanie liczby instancji wybranych usług. Pozwala to zbadać, czy większa elastyczność architektury mikroserwisowej przekłada się na bardziej efektywne wykorzystanie zasobów i lepsze wyniki przy rosnącym obciążeniu.

W ramach eksperymentu analizowany jest wpływ liczby instancji na przepustowość, czas odpowiedzi, zużycie zasobów oraz koszt działania systemu. Wyniki tego scenariusza pozwalają ocenić praktyczną skuteczność skalowania obu badanych architektur.

5.7. Sposób gromadzenia i przetwarzania wyników

Dane pomiarowe uzyskane w trakcie eksperymentów są gromadzone dla każdej serii testowej w sposób uporządkowany i umożliwiający ich dalszą analizę porównawczą. Rejestrowane są zarówno metryki wydajnościowe pochodzące z narzędzi testowych, jak i dane infrastrukturalne związane z wykorzystaniem zasobów oraz konfiguracją środowiska.

W celu zwiększenia wiarygodności wyników każda seria testowa może być wykonywana wielokrotnie, a do dalszej analizy wykorzystywane są wartości uśrednione lub reprezentatywne statystyki opisowe. Takie podejście pozwala ograniczyć wpływ chwilowych odchyleń wynikających z niestabilności środowiska chmurowego lub zakłóceń sieciowych.

Przetwarzanie wyników obejmuje ich agregację, porządkowanie oraz przygotowanie do prezentacji w formie wykresów, tabel i zestawień porównawczych. Na tej podstawie możliwa jest dalsza interpretacja zachowania badanych architektur w różnych scenariuszach testowych.

5.8. Ograniczenia metodyki badawczej

Mimo przyjęcia uporządkowanej i możliwie powtarzalnej metodyki, przeprowadzone badanie posiada pewne ograniczenia. Po pierwsze, eksperyment opiera się na jednej analizowanej aplikacji, co oznacza, że uzyskane wyniki nie muszą być bezpośrednio uogólnialne na wszystkie klasy systemów informatycznych.

Po drugie, na wyniki eksperymentu mogą wpływać cechy konkretnego środowiska chmurowego, w tym sposób przydzielania zasobów, chwilowe obciążenie infrastruktury współdzielonej oraz właściwości sieci. Czynniki te są częściowo ograniczane przez powtarzalność testów, jednak nie mogą zostać całkowicie wyeliminowane.

Po trzecie, przyjęte scenariusze testowe koncentrują się na wybranym punkcie końcowym i określonym modelu obciążenia. Oznacza to, że inne typy żądań lub inna struktura ruchu mogłyby prowadzić do częściowo odmiennych obserwacji. Mimo tych ograniczeń przyjęta metodyka umożliwia przeprowadzenie spójnego i użytecznego porównania badanych architektur.

6. WYNIKI BADAŃ I ICH ANALIZA

W niniejszym rozdziale przedstawiono analizę wyników testów skalowalności przeprowadzonych dla trzech implementacji językowych: Python, JavaScript oraz Java. Badania objęły dwie wersje architektoniczne aplikacji: monolityczną oraz mikroserwisową wykorzystującą gRPC jako technologię komunikacji międzyserwisowej. Analizie poddano dwa mechanizmy zwiększania zasobów: skalowanie wertykalne, polegające na zwiększaniu parametrów pojedynczej instancji maszyny, oraz skalowanie horyzontalne, polegające na zwiększaniu liczby instancji aplikacji lub replik mikroservisów.

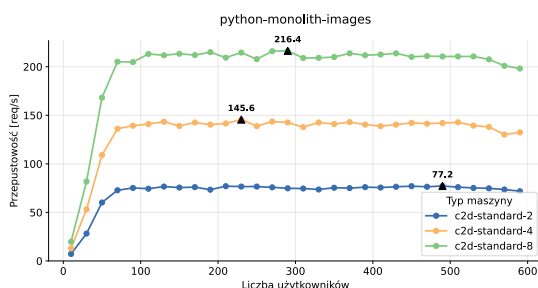
Wyniki zestawiono zarówno w postaci wykresów przedstawiających maksymalną osiąganą przepustowość w funkcji narastającego obciążenia, jak i w tabelach zbiorczych prezentujących maksymalne wartości przepustowości oraz przyrosty wydajności. Celem analizy było określenie, który mechanizm skalowania jest skuteczniejszy dla poszczególnych języków i architektur oraz jak duży wpływ na końcowy rezultat ma sama technologia wykonania aplikacji.

6.1. Wyniki skalowania wertykalnego

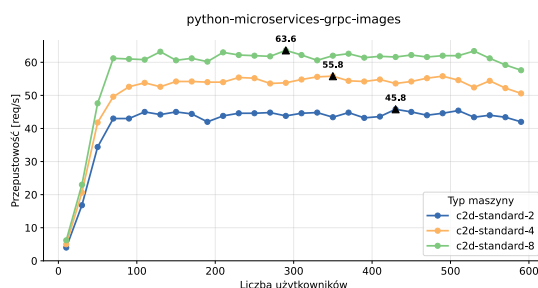
W testach skalowania wertykalnego porównano trzy konfiguracje zasobów obliczeniowych: std-2, std-4 oraz std-8. Oznacza to odpowiednio bazowy poziom zasobów, konfigurację dwukrotnie większą oraz konfigurację czterokrotnie większą względem punktu odniesienia. Tabela 6.1 przedstawia zbiorcze wyniki dla wszystkich badanych implementacji.

Tabela 6.1. Wyniki skalowania wertykalnego dla badanych implementacji.

Lp.	Język	Architektura	Komunikacja	std-2 [req/s]	std-4 [req/s]	std-8 [req/s]	std-4/std-2 [%]	std-8/std-2 [%]	std-8/std-4 [%]
1	Python	monolityczna	-	77,2	145,6	216,4	88,6%	180,3%	48,6%
2	JavaScript	monolityczna	-	1058,8	1096,8	1173,4	3,6%	10,8%	7,0%
3	Java	monolityczna	-	1841,6	3630,6	7275	97,1%	295,0%	100,4%
4	Python	mikroserwisowa	gRPC	45,8	55,8	63,6	21,8%	38,9%	14,0%
5	JavaScript	mikroserwisowa	gRPC	530,4	699,2	946,4	31,8%	78,4%	35,4%
6	Java	mikroserwisowa	gRPC	1354	2523,6	4835,8	86,4%	257,1%	91,6%



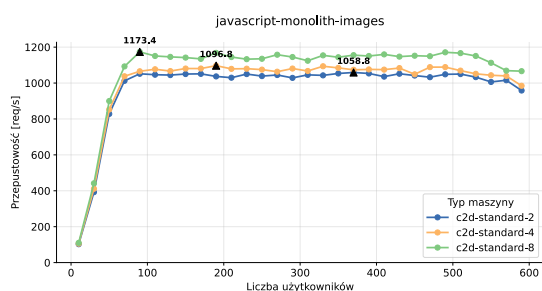
(a) Python – architektura monolityczna



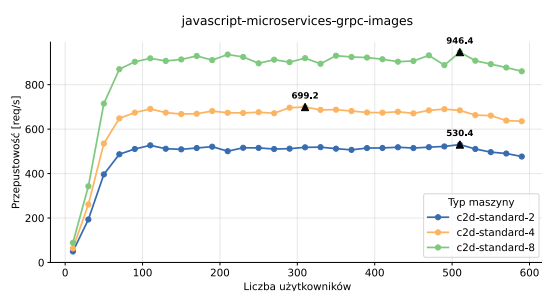
(b) Python – architektura mikroserwisowa z gRPC

Rys. 6.1. Wyniki skalowania wertykalnego dla implementacji w języku Python.

Analiza wyników skalowania wertykalnego pokazuje wyraźnie, że największe korzyści z dokładania zasobów uzyskano dla implementacji w języku Java. W przypadku monolitu przepustowość wzrosła z poziomu 1841,6 req/s dla konfiguracji std-2 do 7275 req/s dla

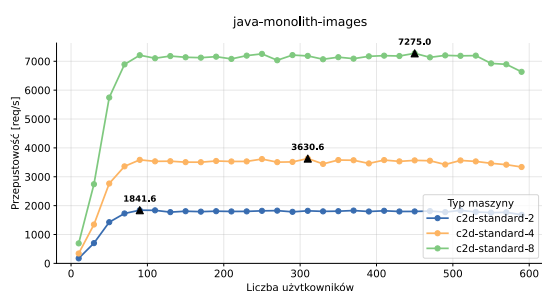


(a) JavaScript – architektura monolityczna

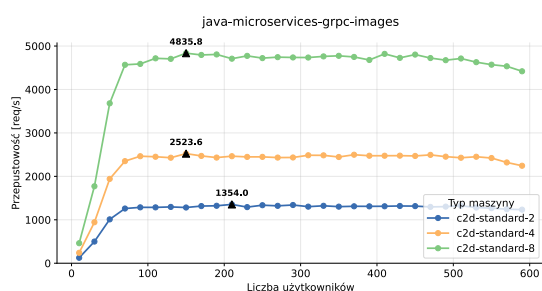


(b) JavaScript – architektura mikroserwisowa z gRPC

Rys. 6.2. Wyniki skalowania wertykalnego dla implementacji w języku JavaScript.



(a) Java – architektura monolityczna



(b) Java – architektura mikroserwisowa z gRPC

Rys. 6.3. Wyniki skalowania wertykalnego dla implementacji w języku Java.

konfiguracji `std-8`, co oznacza przyrost o 295,0%. Bardzo podobny trend zaobserwowano dla mikroserwisów w Javie, gdzie wzrost wyniósł 257,1%. Oznacza to, że implementacja Java efektywnie wykorzystuje dodatkowe rdzenie procesora i pamięć operacyjną, a skalowanie pionowe przekłada się bezpośrednio na wzrost przepustowości.

Odmienne obraz widoczny jest dla JavaScriptu. W architekturze monolitycznej wzrost wydajności po zwiększeniu zasobów był minimalny: z 1058,8 req/s do 1173,4 req/s, czyli jedynie o 10,8%. Sugeruje to, że w tej wersji aplikacji głównym ograniczeniem nie był sam przydział CPU lub RAM, lecz charakter wykonania pojedynczego procesu. Dla mikroserwisów w JavaScript przyrost był wyraźnie większy i osiągnął 78,4%, jednak nadal pozostawał istotnie niższy niż w przypadku Javy.

Python osiągnął najniższe wartości bezwzględne przepustowości, ale sam kierunek zmian był zróżnicowany zależnie od architektury. Monolit zwiększył przepustowość z 77,2 req/s do 216,4 req/s, czyli o 180,3%, natomiast wersja mikroserwisowa jedynie z 45,8 req/s do 63,6 req/s, co daje wzrost rzędu 38,9%. Oznacza to, że w Pythonie skalowanie pionowe poprawia wynik, lecz w przypadku wersji rozproszonej poprawa ta jest ograniczona.

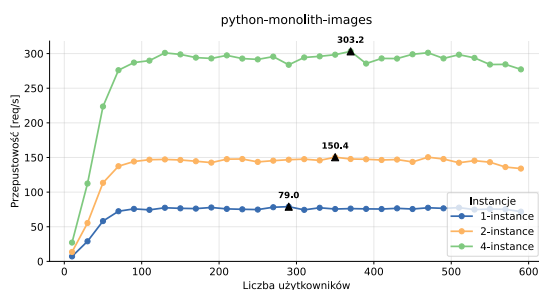
Z punktu widzenia porównania architektur można zauważyć, że dla każdego języka monolit osiągał wyższe wartości maksymalnej przepustowości niż odpowiadająca mu implementacja mikroserwisowa z gRPC. Różnica ta była szczególnie wyraźna dla JavaScriptu oraz Pythona. W przypadku Javy obie architektury reagowały na skalowanie pionowe bardzo dobrze, jednak monolit nadal zachowywał przewagę bezwzględną. Wskazuje to, że dodatkowy narzut komunikacji międzyserwisowej pozostaje istotnym czynnikiem ograniczającym wydajność, nawet wtedy, gdy aplikacja otrzymuje większe zasoby sprzętowe.

6.2. Wyniki skalowania horyzontalnego

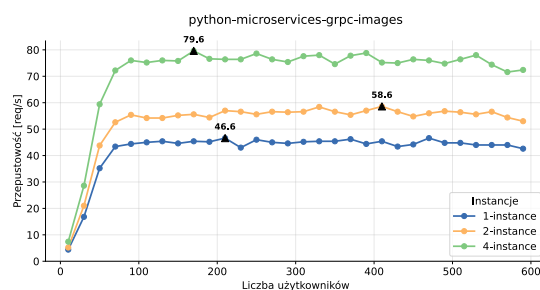
W testach skalowania horyzontalnego analizowano trzy warianty wdrożenia: 1, 2 oraz 4 instancje. W przypadku monolitu oznaczało to uruchomienie wielu kopii całej aplikacji, natomiast w przypadku architektury mikroservisowej zwiększano liczbę replik każdego serwisu. Tabela 6.2 przedstawia zestawienie maksymalnych przepustowości oraz procentowych przyrostów wydajności.

Tabela 6.2. Wyniki skalowania horyzontalnego dla badanych implementacji.

Lp.	Język	Architektura	Komunikacja	1 inst. [req/s]	2 inst. [req/s]	4 inst. [req/s]	2/1 [%]	4/1 [%]	4/2 [%]
1	Python	monolityczna	-	79	150,4	303,2	90,4%	283,8%	101,6%
2	JavaScript	monolityczna	-	1017,4	1385	2318,8	36,1%	127,9%	67,4%
3	Java	monolityczna	-	1817,2	3390	6344,4	86,6%	249,1%	87,2%
4	Python	mikroservisowa	gRPC	46,6	58,6	79,6	25,8%	70,8%	35,8%
5	JavaScript	mikroservisowa	gRPC	559,2	920	1925,6	64,5%	244,3%	109,3%
6	Java	mikroservisowa	gRPC	1355,8	2292,4	5007,8	69,1%	269,4%	118,5%

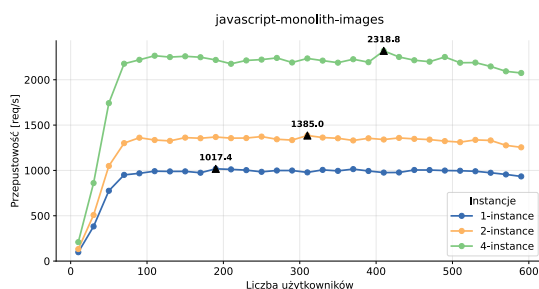


(a) Python – architektura monolityczna

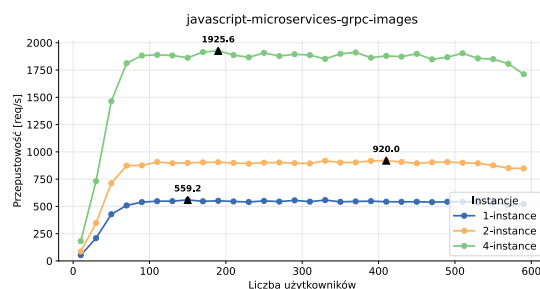


(b) Python – architektura mikroservisowa z gRPC

Rys. 6.4. Wyniki skalowania horyzontalnego dla implementacji w języku Python.



(a) JavaScript – architektura monolityczna

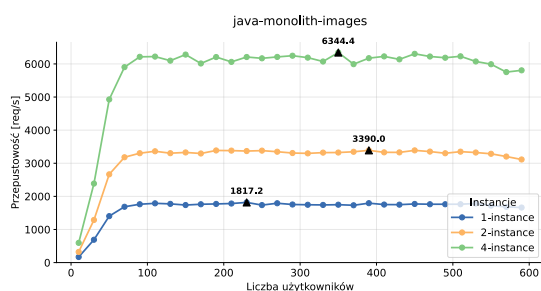


(b) JavaScript – architektura mikroservisowa z gRPC

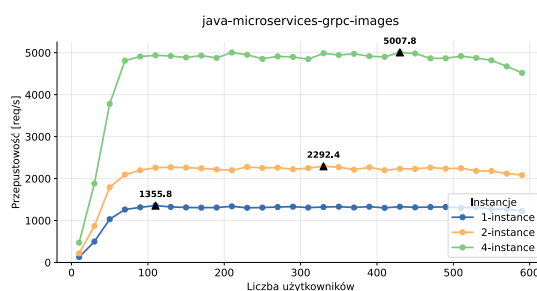
Rys. 6.5. Wyniki skalowania horyzontalnego dla implementacji w języku JavaScript.

Wyniki przedstawione w tabeli 6.2 pokazują, że skalowanie horyzontalne okazało się bardzo skuteczne przede wszystkim dla implementacji Java oraz JavaScript w architekturze mikroservisowej. Największy procentowy wzrost względem konfiguracji bazowej osiągnęła mikroservisowa implementacja w Javie: z 1355,8 req/s do 5007,8 req/s, co oznacza poprawę o 269,4%. Bardzo dobry rezultat uzyskał również monolit w Javie, który zwiększył przepustowość o 249,1%.

W przypadku JavaScriptu szczególnie istotne jest zachowanie wersji mikroservisowej. Przejście z jednej instancji do czterech zwiększyło przepustowość z 559,2 req/s do 1925,6 req/s, czyli o 244,3%. Jest to rezultat zdecydowanie lepszy niż w analogicznym teście skalowania



(a) Java – architektura monolityczna



(b) Java – architektura mikroserwisowa z gRPC

Rys. 6.6. Wyniki skalowania horyzontalnego dla implementacji w języku Java.

wertykalnego. Taki układ wyników wskazuje, że dla tej implementacji bardziej opłacalne jest rozmnażanie instancji niż wzmacnianie pojedynczego węzła.

Python pozostaje najsłabszy pod względem wartości bezwzględnych, jednak także tutaj można zaobserwować poprawę po zwiększaniu liczby instancji. Monolit osiągnął wzrost o 283,8%, z 79 do 303,2 req/s, natomiast wersja mikroserwisowa poprawiła wynik o 70,8%. Oznacza to, że nawet dla technologii o relatywnie niskiej bazowej przepustowości zwiększanie liczby instancji może być skuteczną drogą poprawy wydajności.

Warto zwrócić uwagę na fakt, że przyrosty przy przejściu z 2 do 4 instancji nie zawsze są proporcjonalne do wzrostu liczby replik. Zjawisko to jest naturalne dla systemów rozproszonych, ponieważ wraz z liczbą instancji rośnie również koszt koordynacji, komunikacji oraz równoważenia obciążenia. Mimo tego dla większości badanych przypadków skalowanie horyzontalne pozostawało efektywnym mechanizmem zwiększania przepustowości.

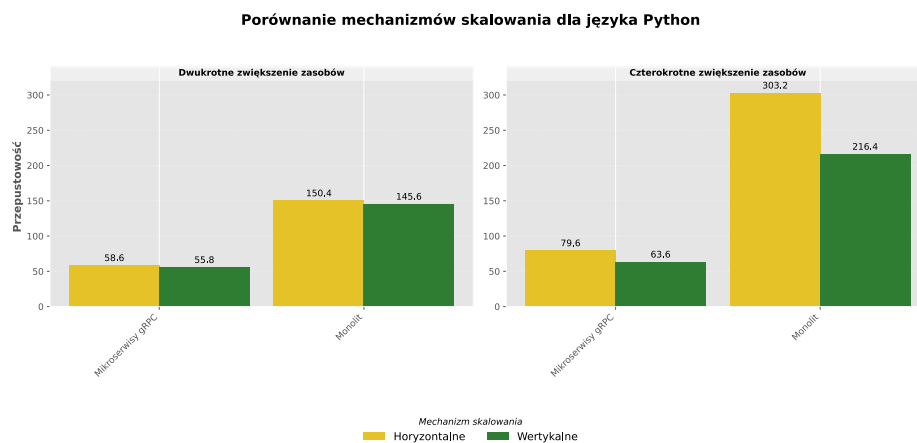
6.3. Porównanie skalowania wertykalnego i horyzontalnego

Aby ocenić, który mechanizm zwiększania zasobów był korzystniejszy, zestawiono ze sobą wyniki dla konfiguracji odpowiadających dwukrotnemu oraz czterokrotnemu wzrostowi zasobów. W praktyce porównano wariant 2 instancji z wariantem std-4 oraz wariant 4 instancji z wariantem std-8. Zestawienie przedstawiono w tabeli 6.3 oraz na rysunkach 6.7–6.9.

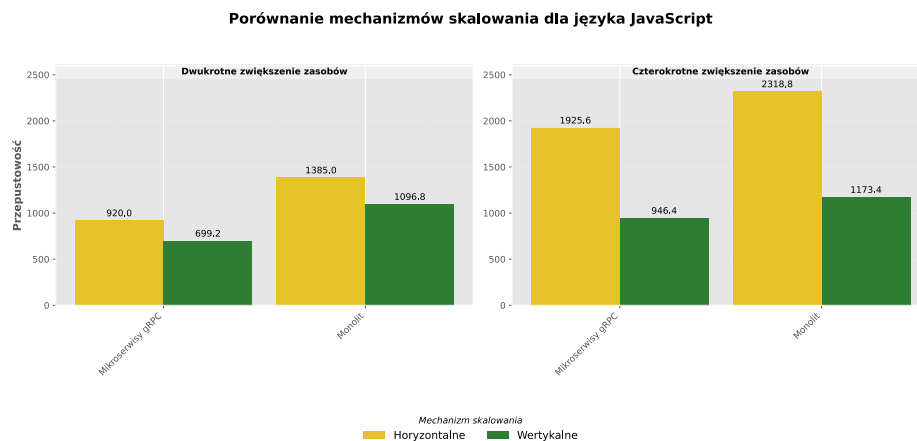
Tabela 6.3. Porównanie skuteczności skalowania horyzontalnego i wertykalnego. Dodatnia wartość oznacza przewagę skalowania wertykalnego, a ujemna przewagę skalowania horyzontalnego.

Lp.	Język	Architektura	Komunikacja	H 2× [req/s]	V 2× [req/s]	V/H 2× [%]	H 4× [req/s]	V 4× [req/s]	V/H 4× [%]
1	Python	monolityczna	-	150,4	145,6	-3,2%	303,2	216,4	-28,6%
2	JavaScript	monolityczna	-	1385	1096,8	-20,8%	2318,8	1173,4	-49,4%
3	Java	monolityczna	-	3390	3630,6	7,1%	6344,4	7275	14,7%
4	Python	mikroserwisowa	gRPC	58,6	55,8	-4,8%	79,6	63,6	-20,1%
5	JavaScript	mikroserwisowa	gRPC	920	699,2	-24,0%	1925,6	946,4	-50,9%
6	Java	mikroserwisowa	gRPC	2292,4	2523,6	10,1%	5007,8	4835,8	-3,4%

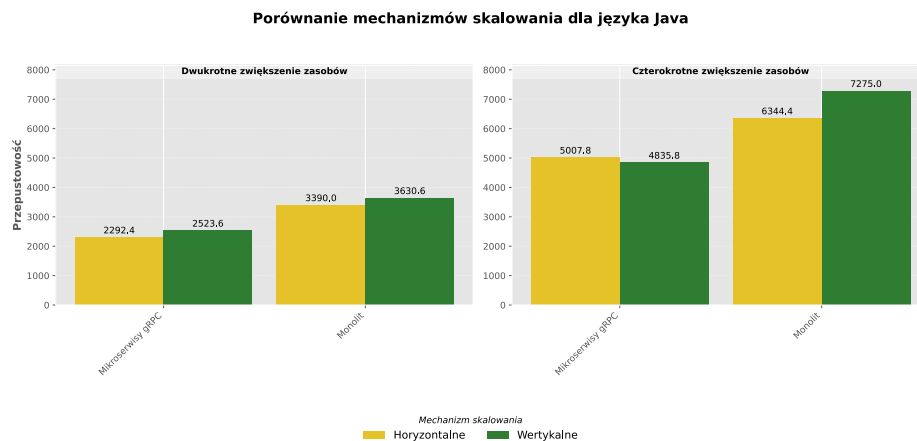
Porównanie obu mechanizmów skalowania prowadzi do jednoznacznego wniosku, że nie istnieje jedno uniwersalne rozwiązanie najlepsze dla wszystkich badanych technologii. W przypadku Pythona i JavaScriptu w każdym analizowanym wariantcie lepsze rezultaty dawało skalowanie horyzontalne. Najbardziej wyraźnie widać to dla JavaScriptu, gdzie dla monolitu przewaga skalowania horyzontalnego nad wertykalnym przy czterokrotnym zwiększeniu zasobów



Rys. 6.7. Porównanie skalowania wertykalnego i horyzontalnego dla implementacji w języku Python.



Rys. 6.8. Porównanie skalowania wertykalnego i horyzontalnego dla implementacji w języku JavaScript.



Rys. 6.9. Porównanie skalowania wertykalnego i horyzontalnego dla implementacji w języku Java.

wyniosła 49,4%, a dla mikroserwisów aż 50,9%. Oznacza to, że w tych implementacjach mnożenie instancji przynosi znacznie lepszy efekt niż wzmacnianie pojedynczego serwera.

Java zachowywała się inaczej. Dla monolitu zarówno przy wzroście zasobów 2×, jak i 4× lepsze wyniki uzyskało skalowanie wertykalne. Przewaga wyniosła odpowiednio 7,1% oraz 14,7%. W przypadku mikroserwisów java sytuacja była bardziej wyrównana: dla konfiguracji 2× lepsze okazało się skalowanie wertykalne, natomiast dla konfiguracji 4× nieznacznie wyższy wynik dało skalowanie horyzontalne. Różnica była jednak niewielka i wyniosła jedynie 3,4% na korzyść skalowania horyzontalnego.

Uzyskane rezultaty sugerują, że dla środowiska java zwiększanie mocy pojedynczej instancji może być bardzo skuteczną strategią, szczególnie dla aplikacji monolitycznych. Z kolei dla Pythona i JavaScriptu bardziej racjonalne wydaje się zwiększanie liczby instancji, ponieważ taka strategia lepiej wykorzystuje charakterystykę tych środowisk uruchomieniowych. W praktyce oznacza to, że decyzja o wyborze sposobu skalowania nie powinna wynikać wyłącznie z architektury systemu, lecz również z właściwości technologii implementacyjnej.

6.4. Podsumowanie wyników

Na podstawie przeprowadzonej analizy można sformułować kilka kluczowych wniosków. Po pierwsze, najwyższą bezwzględną wydajność w badanym zestawie osiągnęły implementacje w języku Java. Dotyczy to zarówno architektury monolitycznej, jak i mikroserwisowej z gRPC. Po drugie, architektura monolityczna we wszystkich językach osiągała wyższą maksymalną przepustowość niż odpowiadająca jej wersja mikroserwisowa, co potwierdza wpływ narzutu komunikacyjnego na wynik końcowy.

Po trzecie, skuteczność skalowania zależała silnie od języka programowania. Java najlepiej reagowała na skalowanie wertykalne, natomiast Python i JavaScript wyraźnie korzystały bardziej ze skalowania horyzontalnego. Szczególnie istotne jest to w przypadku JavaScriptu, dla którego dokładanie CPU i pamięci do pojedynczej instancji dawało mały efekt, natomiast zwiększenie liczby instancji istotnie podnosiło przepustowość.

Ostatecznie wyniki wskazują, że dobór strategii skalowania powinien być podejmowany na podstawie empirycznych pomiarów dla konkretnej technologii i sposobu implementacji. Sama decyzja o wyborze architektury monolitycznej lub mikroserwisowej nie jest wystarczająca. Równie ważne jest to, czy wybrane środowisko uruchomieniowe lepiej wykorzystuje większą liczbę zasobów pojedynczej maszyny, czy też większą liczbę równoległych instancji.

7. ZAGROŻENIA DLA RZETELNOŚCI WYNIKÓW

7.1. Zagrożenia wewnętrzne

Zagrożenia wewnętrzne odnoszą się do tych czynników, które mogą wpływać na uzyskane wyniki w obrębie samego eksperymentu i zaburzać relację pomiędzy badaną przyczyną a obserwowanym skutkiem. W kontekście niniejszej pracy głównym zagrożeniem tego typu jest możliwość, że różnice zaobserwowane pomiędzy wariantem monolitycznym i mikroserwisowym wynikają nie tylko z odmiennego stylu architektonicznego, lecz również z różnic implementacyjnych, konfiguracyjnych lub środowiskowych.

Jednym z kluczowych zagrożeń wewnętrznych jest brak pełnej równoważności obu implementacji. Mimo założenia, że oba warianty realizują ten sam proces biznesowy, każda implementacja może zawierać inne decyzje projektowe, optymalizacje lub uproszczenia, które wpływają na wyniki pomiarów. Ograniczenie tego zagrożenia wymaga zachowania możliwie dużej spójności funkcjonalnej oraz porównywalnego zakresu logiki wykonywanej po obu stronach.

Kolejnym zagrożeniem jest wpływ konfiguracji środowiska uruchomieniowego. Nawet niewielkie różnice w parametrach zasobów, konfiguracji sieci, ustawieniach kontenerów lub sposobie wdrożenia mogą powodować odchylenia w wynikach testów. W związku z tym konieczne jest zachowanie spójnych parametrów środowiska oraz automatyzacja procesu wdrożenia.

Istotnym źródłem zagrożeń wewnętrznych jest również niestabilność środowiska chmurowego. Współdzielenie zasobów z innymi użytkownikami infrastruktury, chwilowe przeciążenia sieci lub zmienność wydajności maszyn wirtualnych mogą wpływać na wyniki eksperymentu niezależnie od badanej architektury. W celu ograniczenia tego wpływu testy powinny być wykonywane wielokrotnie, a analiza powinna opierać się na wynikach zagregowanych.

Do zagrożeń wewnętrznych należy także możliwość błędów pomiarowych. Dotyczy to zarówno błędów związanych z konfiguracją narzędzi testowych, jak i nieprawidłowego gromadzenia lub interpretacji metryk. Aby ograniczyć to ryzyko, konieczne jest stosowanie jednolitej procedury eksperymentalnej i zachowanie tej samej metody pomiaru dla wszystkich badanych wariantów.

7.2. Zagrożenia zewnętrzne

Zagrożenia zewnętrzne dotyczą możliwości uogólnienia uzyskanych wyników poza warunki, w których przeprowadzono badanie. W niniejszej pracy podstawowym ograniczeniem tego rodzaju jest fakt, że analiza została przeprowadzona na jednej konkretnej aplikacji oraz w określonym środowisku chmurowym.

Oznacza to, że uzyskane wyniki nie muszą być bezpośrednio przenoszalne na wszystkie klasy systemów informatycznych. Inne aplikacje mogą różnić się charakterem logiki biznesowej, stopniem złożoności, profilem obciążenia, intensywnością komunikacji między komponentami oraz wymaganiami niefunkcjonalnymi. W rezultacie przewaga jednego stylu architektonicznego zaobserwowana w niniejszym badaniu nie musi występować w identycznej postaci w innym kontekście.

Kolejnym zagrożeniem zewnętrznym jest zależność wyników od konkretnej platformy chmurowej i jej właściwości operacyjnych. Różne środowiska chmurowe mogą stosować odmienne modele przydziału zasobów, rozliczania usług, równoważenia obciążenia oraz komunikacji sieciowej. Z tego względu wyniki uzyskane dla jednego dostawcy chmury nie muszą być w pełni reprezentatywne dla innych środowisk.

Na możliwość uogólnienia wyników wpływa także charakter przyjętych scenariuszy testowych. Eksperyment obejmuje wybrane poziomy obciążenia, określony model ruchu oraz jeden reprezentatywny punkt końcowy aplikacji. W praktyce rzeczywiste systemy mogą być poddawane bardziej złożonym i mniej przewidywalnym wzorcom użycia, co może prowadzić do innych obserwacji.

Mimo tych ograniczeń wyniki badania pozostają wartościowe, ponieważ pozwalają zidentyfikować mechanizmy i zależności, które mogą stanowić punkt odniesienia dla dalszych analiz i kolejnych eksperymentów.

7.3. Zagrożenia konstrukcyjne

Zagrożenia konstrukcyjne odnoszą się do zgodności pomiędzy przyjętymi pojęciami teoretycznymi a sposobem ich operacjonalizacji w eksperymencie. W niniejszej pracy dotyczy to przede wszystkim sposobu rozumienia takich kategorii jak wydajność, skalowalność oraz koszty utrzymania.

Jednym z podstawowych zagrożeń konstrukcyjnych jest możliwość, że wybrane metryki nie oddają w pełni badanego zjawiska. Przykładowo przepustowość i latencja opisują istotne aspekty wydajności, lecz nie wyczerpują całego obrazu jakości działania systemu. Podobnie koszty utrzymania mogą być rozumiane szerzej niż tylko koszt zasobów infrastrukturalnych i obejmować także nakład pracy operacyjnej, złożoność utrzymania czy koszt zarządzania środowiskiem.

Kolejne zagrożenie wynika z przyjęcia jednego punktu końcowego aplikacji jako reprezentatywnego dla całości systemu. Choć podejście to upraszcza analizę i zwiększa porównywalność wyników, może jednocześnie ograniczać pełny obraz zachowania systemu. Inne operacje aplikacyjne mogłyby wykazywać odmienną charakterystykę wydajnościową i skalowalnościową.

Do zagrożeń konstrukcyjnych należy również sposób definiowania kosztów. Jeżeli analiza kosztowa opiera się głównie na kosztach bezpośrednich infrastruktury chmurowej, może nie uwzględniać pełnego kosztu utrzymania systemu, zwłaszcza w przypadku

architektury mikroserwisowej, gdzie istotne znaczenie mają także koszty operacyjne związane z monitorowaniem, wdrażaniem i zarządzaniem większą liczbą komponentów.

Ograniczenie zagrożeń konstrukcyjnych wymaga jasnego zdefiniowania używanych pojęć, konsekwentnego stosowania metryk oraz świadomego wskazania, które elementy rzeczywistości zostały objęte zakresem pomiaru, a które pozostały poza nim.

7.4. Zagrożenia związane z trafnością wnioskowania

Zagrożenia związane z trafnością wnioskowania dotyczą poprawności interpretacji uzyskanych wyników oraz zasadności formułowanych na ich podstawie wniosków. W niniejszej pracy istotne jest przede wszystkim to, aby nie przypisywać nadmiernego znaczenia różnicom obserwowanym w określonych warunkach eksperymentalnych.

Jednym z głównych zagrożeń tego typu jest wyciąganie zbyt szerokich wniosków na podstawie ograniczonego zestawu danych. Nawet jeśli jedna z architektur osiąga lepsze wyniki w określonym scenariuszu, nie oznacza to automatycznie jej ogólnej przewagi we wszystkich warunkach pracy. Interpretacja wyników powinna być zatem osadzona w kontekście konkretnej aplikacji, środowiska i modelu obciążenia.

Innym zagrożeniem jest nieuwzględnienie relacji pomiędzy poszczególnymi metrykami. Przykładowo wyższa przepustowość może być osiągnięta kosztem większej latencji lub znacznie wyższego zużycia zasobów. Podobnie korzystniejsze wyniki skalowalności nie muszą oznaczać lepszej opłacalności ekonomicznej. Wnioski końcowe powinny więc wynikać z łącznej analizy wszystkich obszarów badania, a nie z obserwacji pojedynczej metryki.

Zagrożeniem dla trafności wnioskowania może być również nieuwzględnienie ograniczeń samej metodyki badawczej. Jeżeli ograniczenia te nie zostaną jasno wskazane, istnieje ryzyko nadinterpretacji wyników i przypisania im większej uniwersalności niż rzeczywiście posiadają.

W celu ograniczenia tego rodzaju zagrożeń wnioski formułowane w niniejszej pracy są odnoszone bezpośrednio do przeprowadzonych eksperymentów, zastosowanej konfiguracji środowiska oraz przyjętego zakresu pomiaru. Takie podejście zwiększa przejrzystość interpretacji i pozwala zachować ostrożność metodologiczną.

8. PODSUMOWANIE I WNIOSKI

8.1. *Odpowiedzi na pytania badawcze*

[TWOJE IMIĘ I NAZWISKO]

Głównym celem niniejszej pracy było przeprowadzenie analizy porównawczej wydajności, skalowalności oraz kosztów utrzymania architektury monolitycznej i architektury opartej na mikroservisach w środowiskach chmurowych. W pracy przyjęto, że odpowiedź na główne pytanie badawcze wymaga nie tylko zestawienia wyników wydajnościowych, lecz także uwzględnienia konsekwencji infrastrukturalnych i ekonomicznych związanych z wyborem danego stylu architektonicznego.

Na podstawie przeprowadzonych badań można stwierdzić, że wpływ architektury na zachowanie aplikacji wdrożonej w chmurze nie jest jednoznaczny i zależy od analizowanego aspektu. Architektura monolityczna może zapewniać korzystniejsze właściwości w warunkach bazowych, szczególnie pod względem prostoty przetwarzania i ograniczonego narzutu komunikacyjnego. Z kolei architektura mikroservisowa wykazuje potencjalną przewagę w sytuacjach, w których kluczowe znaczenie ma elastyczne skalowanie wybranych komponentów systemu.

Odpowiedź na główne pytanie badawcze wskazuje zatem, że wybór między architekturą monolityczną a mikroservisową powinien być uzależniony od charakterystyki systemu, oczekiwanego modelu obciążenia, wymagań skalowalnościowych oraz akceptowalnego poziomu złożoności operacyjnej i kosztów utrzymania. Żadne z badanych podejść nie stanowi rozwiązania uniwersalnie najlepszego we wszystkich scenariuszach.

W odniesieniu do pytań pomocniczych można wskazać, że obie architektury różnią się zarówno pod względem zachowania przy wzroście obciążenia, jak i efektywności wykorzystania zasobów. Różnice te stają się szczególnie widoczne po uwzględnieniu mechanizmów skalowania oraz kosztów działania środowiska chmurowego.

8.2. *Najważniejsze wnioski dotyczące wydajności*

Pierwsza grupa wniosków dotyczy wydajności badanych architektur. Uzyskane wyniki wskazują, że architektura monolityczna może osiągać korzystne parametry pracy w warunkach bazowych, szczególnie wtedy, gdy istotne znaczenie ma ograniczenie narzutu komunikacyjnego. Wynika to z faktu, że logika systemu realizowana jest w obrębie jednej aplikacji, bez konieczności wywołań sieciowych pomiędzy usługami.

Architektura mikroservisowa, mimo dodatkowej elastyczności, może generować większe opóźnienia związane z komunikacją pomiędzy komponentami oraz z bardziej złożonym przepływem przetwarzania. Nie oznacza to jednak, że rozwiązanie to zawsze wypada gorzej

wydajnościowo. W określonych warunkach, zwłaszcza przy możliwości izolowania najbardziej obciążonych funkcji, mikroserwisy mogą osiągać satysfakcjonujące lub konkurencyjne wyniki.

Najważniejszy wniosek w obszarze wydajności jest taki, że architektura wpływa nie tylko na wartości samych metryk, lecz także na sposób, w jaki system reaguje na rosnące obciążenie. Z punktu widzenia praktycznego oznacza to, że ocena wydajności nie może opierać się wyłącznie na deklarowanych zaletach danego stylu architektonicznego, lecz musi wynikać z analizy konkretnego przypadku.

8.3. *Najważniejsze wnioski dotyczące skalowalności*

Druga grupa wniosków odnosi się do skalowalności obu badanych wariantów aplikacji. Uzyskane wyniki potwierdzają, że architektura mikroserwisowa lepiej wpisuje się w model elastycznego skalowania charakterystyczny dla środowisk chmurowych. Możliwość niezależnego zwiększania liczby instancji wybranych usług pozwala bardziej precyzyjnie reagować na lokalne przeciążenia i lepiej dopasowywać zasoby do charakterystyki ruchu.

W przypadku architektury monolitycznej skalowanie jest zwykle mniej selektywne, ponieważ dotyczy całej aplikacji. Może to prowadzić do sytuacji, w której wzrost obciążenia jednego fragmentu systemu wymusza zwiększenie zasobów całego rozwiązania. Z drugiej strony prostsza struktura monolitu może oznaczać mniejszy narzut operacyjny i łatwiejsze zarządzanie środowiskiem.

Najważniejszy wniosek dotyczący skalowalności jest taki, że mikroserwisy oferują większą elastyczność, lecz korzyść ta nie jest darmowa. Wiąże się ona ze wzrostem złożoności systemu i wymaga dobrze przygotowanego środowiska uruchomieniowego, obejmującego mechanizmy orkiestracji, monitorowania i automatyzacji.

8.4. *Najważniejsze wnioski dotyczące kosztów utrzymania*

Trzecia grupa wniosków dotyczy kosztów utrzymania systemu w środowisku chmurowym. Przeprowadzona analiza wskazuje, że koszt nie powinien być rozumiany jedynie jako cena zasobów infrastrukturalnych, lecz jako szersza kategoria obejmująca również konsekwencje złożoności architektury, sposobu wdrożenia oraz utrzymania środowiska.

Architektura monolityczna może okazać się korzystniejsza kosztowo w przypadku systemów o umiarkowanej złożoności lub tam, gdzie nie występuje potrzeba niezależnego skalowania wielu komponentów. Prostota wdrożenia i mniejsza liczba elementów środowiska przekładają się w takim przypadku na niższy koszt operacyjny.

Architektura mikroserwisowa może natomiast oferować lepszą efektywność kosztową w scenariuszach, w których selektywne skalowanie rzeczywiście pozwala ograniczyć nadmiarowe zużycie zasobów. Jednocześnie większa liczba usług, kontenerów i mechanizmów zarządzających może powodować wzrost kosztów stałych i operacyjnych.

Najważniejszy wniosek w tym obszarze jest taki, że mikroserwisy nie są automatycznie rozwiązaniem tańszym, a ich opłacalność zależy od rzeczywistego charakteru obciążenia oraz poziomu wykorzystania elastyczności, jaką oferuje to podejście.

8.5. Ograniczenia pracy

Pomimo przyjęcia uporządkowanej metodyki badawczej niniejsza praca posiada pewne ograniczenia. Najważniejszym z nich jest fakt, że analiza została przeprowadzona na jednej konkretnej aplikacji, co ogranicza możliwość pełnego uogólnienia wyników na wszystkie klasy systemów informatycznych.

Kolejne ograniczenie dotyczy środowiska uruchomieniowego. Wyniki eksperymentu zostały uzyskane w określonej konfiguracji infrastruktury chmurowej, a zmiana dostawcy chmury, rodzaju instancji, parametrów sieci lub modelu rozliczania mogłaby wpłynąć na otrzymane rezultaty.

Ograniczeniem jest również zakres scenariuszy testowych, które obejmowały wybrany punkt końcowy aplikacji i określony model obciążenia. Inne scenariusze użycia lub inna struktura ruchu mogłyby prowadzić do częściowo odmiennych obserwacji. Mimo to przyjęte podejście pozwala sformułować użyteczne i dobrze uzasadnione wnioski odnoszące się do badanego przypadku.

8.6. Możliwe kierunki dalszych badań

Przeprowadzone badania mogą stanowić punkt wyjścia do dalszych analiz rozwijających temat porównania architektury monolitycznej i mikroserwisowej. Jednym z naturalnych kierunków dalszych badań jest rozszerzenie eksperymentu na większą liczbę aplikacji o odmiennym charakterze biznesowym, co pozwoliłoby zwiększyć możliwość uogólnienia wyników.

Kolejnym kierunkiem może być analiza większej liczby scenariuszy obciążeniowych, obejmujących różne typy ruchu, zmienność intensywności żądań w czasie oraz bardziej złożone przepływy komunikacji pomiędzy usługami. Pozwoliłoby to lepiej ocenić zachowanie badanych architektur w warunkach bliższych rzeczywistym systemom produkcyjnym.

Warto również rozszerzyć analizę kosztów o dodatkowe elementy, takie jak koszt utrzymania operacyjnego, złożoność monitorowania, czas wdrożeń, niezawodność oraz odporność na awarie częściowe. Szczególnie interesujące może być także porównanie badanych architektur w różnych środowiskach chmurowych lub przy użyciu innych narzędzi orkiestracyjnych.

Dalsze badania mogłyby również objąć analizę wpływu innych czynników technologicznych, takich jak wybór języka programowania, model komunikacji między usługami czy zastosowanie różnych mechanizmów przechowywania danych. Pozwoliłoby to jeszcze pełniej zrozumieć, od czego w praktyce zależy opłacalność i skuteczność danego stylu architektonicznego.

WYKAZ LITERATURY

- [1] Mario Villamizar et al. "Cost comparison of running web applications in the cloud using monolithic, microservice, and AWS Lambda architectures". In: *Service Oriented Computing and Applications* 11 (2017), pp. 233–247. DOI: 10.1007/s11761-017-0208-y. URL: <https://doi.org/10.1007/s11761-017-0208-y>.
- [2] Ioannis Papakonstantinou, Sarantis Kalafatidis, and Lefteris Mamatas. "A Techno-Economic Assessment of Microservices". In: *2020 16th International Conference on Network and Service Management (CNSM)*. IEEE, 2020, pp. 1–5. DOI: 10.23919/CNSM50824.2020.9269114. URL: <https://doi.org/10.23919/CNSM50824.2020.9269114>.
- [3] Freddy Tapia et al. "From Monolithic Systems to Microservices: A Comparative Study of Performance". In: *Applied Sciences* 10.17 (2020), p. 5797. DOI: 10.3390/app10175797. URL: <https://doi.org/10.3390/app10175797>.
- [4] Diego Antonio Castillo Rivas and Daniel Guamán. "Performance and Security Evaluation in Microservices Architecture Using Open Source Containers". In: *Applied Technologies*. Vol. 1388. Communications in Computer and Information Science. Springer, 2021, pp. 484–498. DOI: 10.1007/978-3-030-71503-8_37. URL: https://doi.org/10.1007/978-3-030-71503-8_37.
- [5] Igor Fontana de Nardin et al. "On revisiting energy and performance in microservices applications: A cloud elasticity-driven approach". In: *Parallel Computing* 108 (2021), p. 102858. DOI: 10.1016/j.parco.2021.102858. URL: <https://doi.org/10.1016/j.parco.2021.102858>.
- [6] Grzegorz Blinowski, Anna Ojdowska, and Adam Przybyłek. "Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation". In: *IEEE Access* 10 (2022), pp. 20357–20374. DOI: 10.1109/ACCESS.2022.3152803. URL: <https://doi.org/10.1109/ACCESS.2022.3152803>.
- [7] Victor Benavente et al. "Comparative Analysis of Microservices and Monolithic Architecture". In: *2022 14th International Conference on Computational Intelligence and Communication Networks (CICN)*. IEEE, 2022, pp. 177–184. DOI: 10.1109/CICN56167.2022.10008275. URL: <https://doi.org/10.1109/CICN56167.2022.10008275>.
- [8] Poonam Mangwani and Vrinda Tokekar. "Container Based Scalability and Performance Analysis of Multitenant SaaS Applications". In: *2022 13th International Conference on Computing Communication and Networking Technologies (ICCCNT)*. IEEE, 2022, pp. 1–6. DOI: 10.1109/ICCCNT54827.2022.9984214. URL: <https://doi.org/10.1109/ICCCNT54827.2022.9984214>.
- [9] Szymon Okrój and Przemysław Jatkiewicz. "Differences in performance, scalability, and cost of using microservice and monolithic architecture". In: *Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing (SAC '23)*. New York, NY, USA: Association for Computing Machinery, 2023, pp. 1038–1041. DOI: 10.1145/3555776.3578725. URL: <https://doi.org/10.1145/3555776.3578725>.

- [10] Poonam Mangwani, Niti Mangwani, and Sachin Motwani. "Evaluation of a Multitenant SaaS Using Monolithic and Microservice Architectures". In: *SN Computer Science* 4.2 (2023), p. 185. DOI: 10.1007/s42979-022-01610-2. URL: <https://doi.org/10.1007/s42979-022-01610-2>.
- [11] Kendricko Adrio et al. "Comparative Analysis of Monolith, Microservice API Gateway and Microservice Federated Gateway on Web-based application using GraphQL API". In: *2023 10th International Conference on Electrical Engineering, Computer Science and Informatics (EECSI)*. IEEE, 2023. DOI: 10.1109/EECSI59885.2023.10295809. URL: <https://doi.org/10.1109/EECSI59885.2023.10295809>.
- [12] Nahid Nawal Nayim et al. "Performance Evaluation of Monolithic and Microservice Architecture for an E-commerce Startup". In: *2023 26th International Conference on Computer and Information Technology (ICCIT)*. IEEE, 2023, pp. 1–5. DOI: 10.1109/ICCIT60459.2023.10441241. URL: <https://doi.org/10.1109/ICCIT60459.2023.10441241>.
- [13] M. S. Rashmi et al. "Synthetic to Real-World: Insights on Microservices-Based Application Benchmarks". In: *2024 IEEE International Conference on Information Technology, Electronics and Intelligent Communication Systems (ICITEICS)*. IEEE, 2024. DOI: 10.1109/ICITEICS61368.2024.10625221. URL: <https://doi.org/10.1109/ICITEICS61368.2024.10625221>.
- [14] Vincent Berry et al. "Is it Worth Migrating a Monolith to Microservices? An Experience Report on Performance, Availability and Energy Usage". In: *2024 IEEE International Conference on Web Services (ICWS)*. IEEE, 2024, pp. 944–954. DOI: 10.1109/ICWS62655.2024.00112. URL: <https://doi.org/10.1109/ICWS62655.2024.00112>.
- [15] Jurayak Kader et al. "Comparative Analysis between Monolithic and Microservice Architecture Based on Web Streaming Applications". In: *2025 International Conference on Electrical, Computer and Communication Engineering (ECCE)*. IEEE, 2025. DOI: 10.1109/ECCE64574.2025.11014023. URL: <https://doi.org/10.1109/ECCE64574.2025.11014023>.
- [16] Reinhard Bernsteiner et al. "Performance Comparison between a Monolithic and a Microservice Application". In: *Proceedings of the Workshop on Software Quality, Analysis, Monitoring, Improvement, and Applications (SQAMIA 2025)*. Vol. 4077. CEUR Workshop Proceedings. 2025, pp. 1–8. URL: <https://ceur-ws.org/Vol-4077/paper2.pdf>.

WYKAZ RYSUNKÓW

3.1	Liczba wybranych publikacji w zależności od roku wydania	19
3.2	Udział poszczególnych źródeł publikacyjnych w końcowym zbiorze	20
4.1	Schemat infrastruktury wariantu monolitycznego	27
4.2	Diagram sekwencji przetwarzania w wariacie monolitycznym	28
4.3	Schemat infrastruktury wariantu mikroserwisowego	28
4.4	Diagram sekwencji przetwarzania w wariacie mikroserwisowym	29
6.1	Wyniki skalowania wertykalnego dla implementacji w języku Python.	37
6.2	Wyniki skalowania wertykalnego dla implementacji w języku JavaScript.	38
6.3	Wyniki skalowania wertykalnego dla implementacji w języku Java.	38
6.4	Wyniki skalowania horyzontalnego dla implementacji w języku Python.	39
6.5	Wyniki skalowania horyzontalnego dla implementacji w języku JavaScript.	39
6.6	Wyniki skalowania horyzontalnego dla implementacji w języku Java.	40
6.7	Porównanie skalowania wertykalnego i horyzontalnego dla implementacji w języku Python.	41
6.8	Porównanie skalowania wertykalnego i horyzontalnego dla implementacji w języku JavaScript.	41
6.9	Porównanie skalowania wertykalnego i horyzontalnego dla implementacji w języku Java.	41

WYKAZ TABEL

6.1	Wyniki skalowania wertykalnego dla badanych implementacji.	37
6.2	Wyniki skalowania horyzontalnego dla badanych implementacji.	39
6.3	Porównanie skuteczności skalowania horyzontalnego i wertykalnego. Dodatnia wartość oznacza przewagę skalowania wertykalnego, a ujemna przewagę skalowania horyzontalnego.	40

WYKAZ LISTINGÓW

4.1	Przykładowe wywołanie punktu końcowego HTTP	24
4.2	Logiczny model żądania wyszukiwania	24
4.3	Przykładowa odpowiedź systemu	24